Confidence-Aware Ontology-Based Inference for Missing Edge Prediction in Knowledge Graphs

Rule-Based Knowledge Graph Completion on the CoDEx-M Benchmark

Phase 1 Project Report: Literature Review and Implementation Progress

Master of Technology in Computer Science and Engineering with
Specialization in Data Science and Artificial Intelligence

by

SUGITHRA C S

25021396

Under the Guidance of

Dr. Shailesh Sivan
Assistant Professor
Department of Computer Science, CUSAT



DEPARTMENT OF COMPUTER SCIENCE
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

November 2026

DECLARATION

I, Sugithra C S (Register Number: 25021396), hereby declare that this project report entitled **Confidence-Aware Ontology-Based Inference for Missing Edge Prediction in Knowledge Graphs**, submitted to the Department of Computer Science, Cochin University of Science and Technology, in partial fulfilment of the requirements for the degree of Master of Technology in Computer Science and Engineering with Specialization in Data Science and Artificial Intelligence, is a record of original work carried out by me under the guidance of Dr. Shailesh Sivan, Assistant Professor, Department of Computer Science, CUSAT.

I further declare that this work has not been submitted elsewhere for the award of any degree, diploma, or other similar title, and that all sources of information and material used in this report have been duly acknowledged and properly cited.

Ernakulam-682022
November 2026

Sugithra C S
(Register No: 25021396)
Department of Computer Science
CUSAT

Student Signature: _____

Dr. Shailesh Sivan
Assistant Professor
Department of Computer Science, CUSAT

Guide Signature: _____

Abstract

Knowledge graphs are incomplete, and rule-based completion offers predictions that can be traced to the facts that support them. This Phase 1 report reviews 43 unique research papers on knowledge graph completion, logical rule learning, rule confidence, ontology-based reasoning, explainability, embeddings, graph neural networks and evaluation, and describes the rule-based completion system implemented on the CoDEx-M benchmark (185,584 training triples, 17,050 entities and 51 relations). The pipeline verifies the dataset splits, builds a directed relation-labelled training graph, serialises the training triples as an RDF/OWL dataset in TriG with separate assertion, label and provenance graphs, and mines one- and two-atom rules with support, standard confidence, PCA confidence, head coverage and a Wilson lower bound. A frozen set of 160 rules, reviewed on the validation split, generates 184,487 unique candidate triples that are absent from training. Each candidate is scored by the highest Wilson lower bound of its supporting rules and explained by its rule groundings; ranking V1 and V2 differ only in their tie-breaks (V2 prefers more distinct training proofs). An interactive prediction explorer and a cap-comparison dashboard present the results. Per-relation caps of 100, 250, 500 and 1,000 were compared on the validation split with a selection rule frozen in advance, and the selected configuration (cap 1,000, V2) was evaluated on the test split: it makes 23,772 predictions, of which 927 match test positives and 73 match supplied negatives, giving a benchmark precision of 92.70%, a recall of 8.99%, an F1-score of 16.39% and a positive hit rate of 3.90%; the cap-100 V2 baseline gives 278 matches, 95.53% precision and 2.70% recall. The 22,772 unlabelled predictions are treated as unknown, not false. The test split was used twice (cap-100 baseline, then the selected configuration), the ontology is generated but does not influence the score, and no ranking metric or embedding baseline has been run. The contribution is a transparent, reproducible rule-based baseline with documented protocols and a literature-based plan for ontology-aware scoring, ranked evaluation and baseline comparison.

Keywords: knowledge graph completion; missing-edge prediction; logical rule mining; PCA confidence; Wilson lower bound; ontology; RDF/OWL; explainability; open-world assumption; CoDEx-M

ACKNOWLEDGEMENT

I am grateful to the Department of Computer Science, Cochin University of Science and Technology, for providing the academic foundation and research environment that enabled the progress of this project. The coursework, laboratory exposure, and technical guidance I received from the department contributed significantly to the methodological and analytical skills applied throughout this work.

I would like to express my sincere gratitude to my project guide, Dr. Shailesh Sivan, Assistant Professor, Department of Computer Science, for his constant support, insightful feedback, and expert supervision. His guidance has been instrumental in shaping the research direction, deepening my understanding, and improving the overall quality of this work.

I also express my sincere gratitude to Dr. Bijoy A. Jose, Associate Professor and Head of the Department, for his support and encouragement throughout this project.

I would also like to thank the faculty members of the Department of Computer Science for their encouragement, as well as the broader knowledge graph and machine learning research community whose published work on knowledge graph embedding, reasoning, and rule learning provided valuable insights that inspired this project.

Finally, I express my heartfelt gratitude to my family and friends for their continuous encouragement, patience, and unwavering support throughout this research journey.

Contents

Declaration i

Abstract ii

Acknowledgement iii

Abbreviations ix

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Key terms . . . 2
- 1.3 Organisation of the report . . . 3

2 Literature Review 4
- 2.1 Scope, eligibility and categorisation criteria . . . 4
- 2.2 Rule-based knowledge graph completion and logical rule learning . . . 5
- 2.3 Rule confidence and candidate-specific scoring . . . 6
- 2.4 Ontology-based reasoning and semantic constraints . . . 7
- 2.5 Explainable knowledge graph inference . . . 8
- 2.6 Neural-symbolic reasoning and differentiable rule learning . . . 9
- 2.7 Knowledge graph embeddings and link prediction . . . 10
- 2.8 Graph neural networks and advanced learning methods . . . 11
- 2.9 Knowledge graph surveys, benchmarks and evaluation . . . 11
- 2.10 Development of the field . . . 13
- 2.11 Key lessons from the literature . . . 13
- 2.12 Research gap analysis . . . 13

3 Project Overview, Problem Statement and Objectives 15
- 3.1 Problem statement . . . 15
- 3.2 Objectives . . . 15
- 3.3 Original proposal and its realisation . . . 16
- 3.4 Scope and expected contribution . . . 16
- 3.5 Research questions . . . 17
- 3.6 Comparative analysis . . . 17

4 Proposed and Implemented Methodology 18
- 4.1 Published approaches and the conceptual pipeline . . . 18
- 4.2 Architecture of the implemented system . . . 18
- 4.3 Data and splits . . . 20
- 4.4 Graph construction and visualisation . . . 20
- 4.5 RDF/OWL ontology generation . . . 21
- 4.6 Rule mining and validation review . . . 22
- 4.7 Candidate generation, scoring and V1/V2 ranking . . . 24
- 4.8 Worked examples from the saved results . . . 24
- 4.9 Prediction explorer . . . 24

4.10 Evaluation protocol and metrics . . . 25
4.11 Cap experiment and validation-based selection . . . 26

5 Implementation, Experiments and Evaluation 28
5.1 Tools and implementation status . . . 28
5.2 Rule mining results . . . 28
5.3 Candidate generation results . . . 29
5.4 Baseline test evaluation at cap 100 (Phase 6) . . . 29
5.5 Cap experiment on the validation split . . . 29
5.6 Final test evaluation of the selected configuration . . . 30
5.7 Discussion . . . 30
5.8 Verification and reproducibility . . . 32

6 Timeline, Milestones and Deliverables 34
6.1 Gantt chart . . . 34
6.2 Milestones and deliverables . . . 35

7 Conclusion, Limitations and Future Work 36
7.1 Conclusion . . . 36
7.2 Limitations . . . 36
7.3 Future work . . . 37

References 38

A Repository and Reproducibility Notes 42
A.1 Repository structure . . . 42
A.2 Running the pipeline . . . 42
A.3 Output files used in this report . . . 43
A.4 Reproducibility measures . . . 43

List of Figures

1.1 Two-hop rule inference with genuine CoDEx-M facts 1

2.1 Taxonomy of the 43 reviewed papers by research theme, with the principal methods in each theme and the implementation status of the theme in this project († = selected paper). 5

4.1 Architecture of the Proposed Rule-Based Knowledge Graph Completion System Using CoDEx-M. Green arrows: training-only data flow; amber dashed arrows: validation use (rule review, cap selection); red dashed arrows: test use (cap-100 baseline first, selected cap-1,000 V2 configuration second); dashed grey box: proposed extensions that are not implemented. The ontology does not participate in scoring. 19

4.2 The 20 most frequent relations in the CoDEx-M training graph (`graph_statistics.json`). 20

4.3 Screenshots of the Phase 2 training-graph explorer: entity search and relationship panel (top) and the drawn one-hop neighbourhood of Katherine Jackson (bottom). Only training triples are shown. 21

4.4 Visualisation of the triples of Listing 4.1 by named graph. The graphs are kept apart so that a consumer selects the training assertions explicitly. 22

4.5 Prediction explorer, default graph view: the 250 best-ranked of 3,648 predicted triples; dashed edges are predictions coloured by score band (green ≥ 0.7, orange 0.4–0.7, red < 0.4). 25

4.6 Entity search and entity view for Katherine Jackson: 15 training edges (solid) and 2 predicted edges (dashed): spouse $\rightarrow$ Joe Jackson (score 0.918) and place of burial $\rightarrow$ Forest Lawn Memorial Park (score 0.377). 25

4.7 Evidence view and provenance panel for the prediction (Katherine Jackson, spouse, Joe Jackson): both supporting rules with their statistics and the training groundings drawn as solid edges. 26

4.8 Prediction table filtered to V2 and spouse (P26), sorted by rank (left), and the V1-versus-V2 comparison tab showing 2,830 predictions kept in both, 409 dropped, 409 added and 1,499 with a changed rank (right). 26

5.1 Validation metrics across per-relation caps for V1 and V2, plotted from `validation_results.json`. Benchmark precision falls slightly and the positive hit rate falls as the cap grows, while matches, recall and F1 rise. 30

5.2 Screenshots of the cap-comparison dashboard: summary of the selected configuration (top), validation charts for caps 100–1,000 with the selected configuration ringed (middle), and the complete validation comparison table (bottom). 31

5.3 Test-split comparison of the three evaluated configurations, plotted from `evaluation_results.json` (cap 100) and `test_results.json` (cap 1,000 V2). 32

5.4 Test table of the cap-comparison dashboard; the values agree with Table 5.4. . . . 33

5.5 Positive and supplied-negative test matches of cap 1,000 V2 for the 15 relations with the most matches (`test_final/per_relation_metrics.csv`). 33

6.1 Gantt chart. Dark green: completed, placed from dated project files; light green: completed, period approximate; amber: in progress; blue: planned (relative months, dates to be confirmed). The red line marks the end of Phase 1. 34

A.1 Phase 6 evaluation tab of the prediction explorer: inputs with SHA-256 prefixes and the cap-100 V1/V2 test metrics. 44

List of Tables

1.1 Key terms used in this report. 2

2.1 The five selected papers (files in the `selected five` folder, matched by SHA-256 hash to the main archive). 4
2.2 Comparative review: rule-based knowledge graph completion and logical rule learning. 6
2.3 Comparative review: rule confidence and candidate-specific scoring. 7
2.4 Comparative review: ontology-based reasoning and semantic constraints. 8
2.5 Comparative review: explainable knowledge graph inference. 9
2.6 Comparative review: neural-symbolic reasoning and differentiable rule learning. . . 10
2.7 Comparative review: embeddings, graph neural networks, surveys, benchmarks and evaluation. 11
2.8 Development of the main reviewed approaches by year. 13
2.9 Research gap analysis: gaps supported by the literature and their status in this project. 14

3.1 Objectives, status, evidence and planned extensions. 15
3.2 Elements of the original proposal compared with the implementation. 16

4.1 Conceptual pipeline stages in published approaches and in the implemented system. 18
4.2 CoDEx-M files, verified from `outputs/phase1_dataset_report.md`. Entities and relations are those occurring in each file. 20
4.3 Genuine rules from the frozen 160-rule set (`run_grid_ext/selected_rules.json`). Supp. = support; Body = body size; Std = standard confidence. Wilson bounds in the set range from 0.079 to 0.957. 23

5.1 Implementation status of the pipeline stages. 28
5.2 Test evaluation of the cap-100 baseline (`outputs/phase6/evaluation_test/evaluation_results.json`). 29
5.3 Validation comparison of the eight configurations (`outputs/cap_experiment/validation/validation_results.json`); recall over 10,310 validation positives. Prec. lower = Wilson 95% lower bound. Highlighted: the configuration selected by rule A1. 30
5.4 Test results of the cap-100 baseline (Phase 6) and of the selected cap-1,000 V2 configuration (second use of the test split). 32

6.1 Gantt tasks, status and the dated evidence used to place them (times in UTC, from the manifests). 34
6.2 Milestones and deliverables. 35

A.1 Output files used in this report and their content. 43

Abbreviations

Abbreviation	Meaning
KG	Knowledge graph
KGC	Knowledge graph completion
CoDEx-M	Medium-sized CoDEx benchmark derived from Wikidata and Wikipedia [44]
RDF	Resource Description Framework
OWL	Web Ontology Language
TriG	RDF serialisation format that supports named graphs
OWA	Open-world assumption
PCA	Partial completeness assumption (PCA confidence)
TP / FP / FN	Matches to test positives / matches to supplied negatives / unpredicted positives
V1 / V2	Ranking version 1 / version 2 (different tie-breaks of the same score)
MRR	Mean reciprocal rank
Hits@K	Share of queries whose correct answer is among the top K ranked answers
GNN / GCN	Graph neural network / graph convolutional network
EM	Expectation maximisation
F1	Harmonic mean of precision and recall
UI	User interface

1 Introduction

A knowledge graph (KG) stores facts as directed, labelled links between entities. Each fact is a triple (head, relation, tail), also written $r(h, t)$; in the Resource Description Framework (RDF) it is a subject–predicate–object triple. KGs support search, question answering, recommendation and data integration [36, 37]. Real KGs are built largely by automated extraction and are never complete [1, 6]: if a triple is absent, the graph does not say that it is false; it only fails to record it. Predicting absent but plausible triples is called knowledge graph completion, link prediction or missing-edge prediction [35, 36, 38].

A logical rule expresses a regular pattern in the graph. Figure 1.1 uses two genuine training facts of the CoDEx-M benchmark used in this project. The rule educatedAt$(X, Y) \wedge$ country$(Y, Z) \Rightarrow$ citizenOf(X, Z) was mined from the training graph, and applied to the two facts it derives the candidate (Jeremy Piven, country of citizenship, United States of America). The candidate is not in the training graph, it is a triple of the held-out test split, and it can be traced to its rule and to the facts that produced it [1, 4, 12].

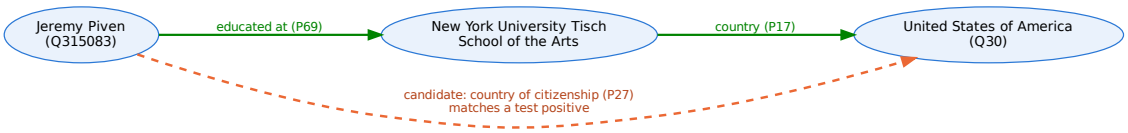


Figure 1.1: Two-hop rule inference with genuine CoDEx-M facts. Solid edges: training triples (`data/train.txt`); dashed edge: the candidate derived by the mined rule P69$(x, y) \wedge$ P17$(y, z) \Rightarrow$ P27(x, z) (PCA confidence 0.841, Wilson lower bound 0.810, support 543). The candidate is a test positive; it was predicted from training data only.

Real rules are not always true, so each carries a confidence: the proportion of body matches for which the head also holds. Wu et al. [2] and Zhang et al. [6] note that such statistics can mislead when the graph is incomplete, and Evoda [2] therefore adopts the PCA confidence. An ontology defines classes, relation types and constraints such as domains and ranges, and ontology-based reasoning is one of the main groups of KG reasoning [33]; it has been used to make completion more consistent [7]. In this project an RDF/OWL representation of the training graph is generated, but it does not yet influence the prediction score (Chapter 4). Explanations matter as well: explainability is a stated challenge for approaches that integrate logics with embeddings [6] and a motivation for path-based completion [10].

1.1 Motivation

Incomplete graphs. Missing relationships limit downstream reasoning and knowledge discovery [6, 35]. In CoDEx-M, the validation and test splits hold 20,621 positive triples that are not in the training graph (Section 4.3).

Explanations beyond embeddings. Embedding models are strong link predictors [19, 20, 38], but surveys note that embeddings can lack interpretability and encode biases [40], that integrated logic-and-embedding methods often remain black boxes [6], and that symbolic rule learning gives transparent paths [43]. These statements concern the surveyed methods in general;

some work mines rules from embeddings [17]. Rule-based inference offers a direct way to justify a link by supporting facts [1, 12].

Global rule confidence versus candidate evidence. A rule usually has one weight for the whole graph, which ignores the query instance [4]: two candidates from the same rule share a confidence although one may rest on many paths and the other on one. The implemented system scores with a global rule statistic and uses the number of distinct training proofs only as a tie-break (ranking V2).

Open-world uncertainty. A missing triple may be false or merely unobserved [34]. A prediction that matches neither a held-out positive nor a supplied negative is therefore *unlabelled*, not an error, and every precision figure in this report is computed only over labelled predictions.

Ontologies and transparent evaluation. An ontology could check the semantic compatibility of candidates [7–9]; this is a hypothesis for future work, and constraints do not automatically improve accuracy. Benchmarks do not guarantee diverse logic patterns and their silver-standard correctness is questioned [6], and tied scores can make naive rankings optimistic [4], so evaluation needs a documented split protocol and verified numbers (Sections 4.10 and 5.6).

Reliable and interpretable completion matters where wrong inferences are costly, such as healthcare data integration, industrial fault prediction and responsible AI [9, 11, 32]. Users need to know why a link was proposed, how far it can be trusted and how it was evaluated.

1.2 Key terms

Table 1.1 explains the terms used throughout the report.

Table 1.1: Key terms used in this report.

Term	Meaning in this report	Used in
Triple	A fact (subject, relation, object) with Wikidata-style IDs, e.g. (Q315083, P27, Q30)	Ch. 4
Rule	A pattern body ⇒ head with one or two body atoms, e.g. $\text{P26}(z, x) \Rightarrow \text{P26}(x, z)$	Sec. 4.6
Support	Number of distinct body pairs (or couples) for which the head is also a training triple	Sec. 4.6
Standard / PCA confidence	Support divided by all body pairs / by body pairs whose subject (or object) already has some head fact	Sec. 4.6
Wilson lower bound	95% lower confidence bound of the PCA confidence; penalises rules with little evidence	Sec. 4.6
Candidate	A triple derived by a rule from training facts that is not itself a training triple	Sec. 4.7
Grounding / body match	The training triples that make a rule body true for a candidate	Sec. 4.7
Cap	The maximum number of ranked predictions kept for each relation	Sec. 4.11
Supplied negative	A triple from `valid_negatives.txt` or `test_negatives.txt`, distributed with CoDEx-M as a negative example	Sec. 4.10
Unlabelled prediction	A prediction that is neither a held-out positive nor a supplied negative; not counted as false	Sec. 4.10
Benchmark precision	TP/(TP+FP), computed over labelled predictions only	Sec. 4.10

1.3 Organisation of the report

Chapter 2 gives the thematic literature review of 43 papers and the research-gap analysis, Chapter 3 the problem statement, objectives and research questions, Chapter 4 the implemented methodology and architecture, Chapter 5 the implementation, experiments and verified results, Chapter 6 the timeline, milestones and deliverables, and Chapter 7 the conclusion, limitations and future work. Appendix A lists the repository structure and reproducibility instructions.

2 Literature Review

2.1 Scope, eligibility and categorisation criteria

Eligibility criteria. A paper was included if it (i) was in the supplied archive `project research papers KG new.zip`, (ii) concerns knowledge graph completion, reasoning, rule learning, ontology-based reasoning, explainability, or the embedding, graph-learning and evaluation work on which these depend, (iii) is a unique paper, and (iv) has bibliographic details that could be checked against its PDF. The archive holds 50 PDF files: 45 in the main folder and 5 in the sub-folder `selected five`. All five files in `selected five` are byte-identical copies of files in the main folder. The main folder also contains two byte-identical copies of *Anytime bottom-up rule learning for large-scale knowledge graph completion*, and two versions of the Cao et al. survey (the published ACM version and the author-deposited version of the same article). Removing these duplicates leaves **43 unique papers**. All 43 are cited. The only reference outside the archive is the CoDEx dataset paper [44], which is cited as the source of the benchmark and is not part of the reviewed set. Of the 43 papers, 41 are peer-reviewed and 2 are arXiv preprints [4, 5].

Selected papers. The five papers of the `selected five` folder are the core references of the project (Table 2.1); they are marked with † throughout the chapter. They are discussed in their thematic sections, so the review keeps a single thematic order rather than placing them in a separate block.

Table 2.1: The five selected papers (files in the `selected five` folder, matched by SHA-256 hash to the main archive).

No.	Ref.	Exact title (authors, year)	Theme
S1	[2]	Rule learning over knowledge graphs with genetic logic programming (Wu et al., 2022)	Rule-based KGC and rule learning
S2	[4]	SLogic: Subgraph-Informed Logical Rule Learning for Knowledge Graph Completion (Le et al., 2025)	Rule confidence and candidate-specific scoring
S3	[5]	Probabilistic Circuits for Knowledge Graph Completion with Reduced Rule Sets (Patil et al., 2025)	Rule confidence and candidate-specific scoring
S4	[12]	Differentiable Learning of Logical Rules for Knowledge Base Reasoning (Yang, Yang and Cohen, 2017)	Neural-symbolic and differentiable rule learning
S5	[39]	Knowledge Graph Embedding: A Survey from the Perspective of Representation Spaces (Cao et al., 2024)	Surveys, benchmarks and evaluation

Depth of reading. Nine papers [1,2,4–6,12–15] were read in targeted full-text sections, three more [3, 7, 10] at the level of abstract, definitions, datasets and conclusion, and 31 at the level of abstract, introduction and key sections. Limitations are given only where the text supports them; limitations *stated* by the authors are separated from observations made in this review (marked *Own*).

Categorisation criteria. Each paper is assigned to one theme by its principal method and its role in missing-edge prediction: how rules are discovered; how a candidate is scored or combined; whether ontology semantics influence reasoning; whether the justification of a prediction is a

design goal; whether rules or weights are learned by neural methods; whether triples are scored by embeddings; whether graph neighbourhoods are learned; or whether the paper defines surveys, benchmarks and evaluation. Themes, and the papers within them, are ordered by relevance to the project, from rule-based completion towards general background. Figure 2.1 shows the taxonomy and how far each theme is reflected in the current CoDEx-M implementation. Each theme ends with a comparative table whose last row positions this project.

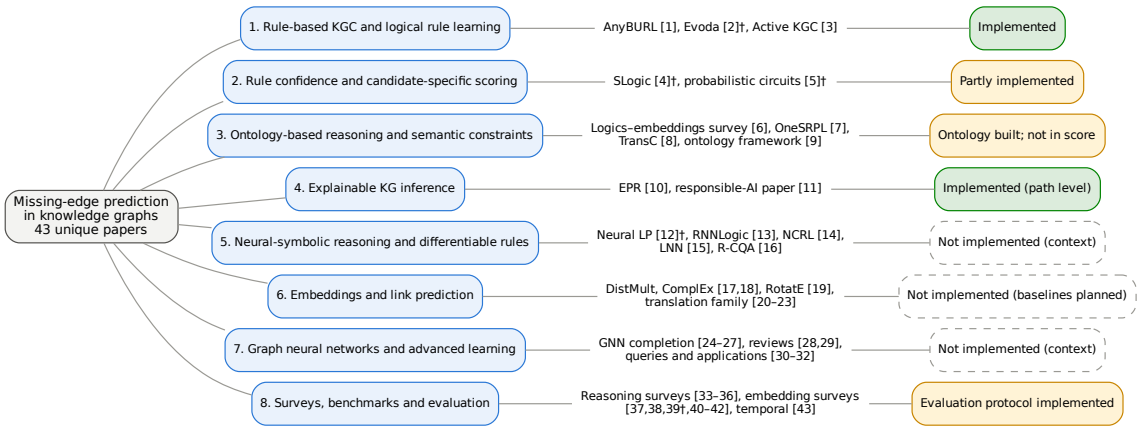


Figure 2.1: Taxonomy of the 43 reviewed papers by research theme, with the principal methods in each theme and the implementation status of the theme in this project († = selected paper).

2.2 Rule-based knowledge graph completion and logical rule learning

Grouping criterion: papers whose principal contribution is how logical rules are discovered and given a confidence for missing-edge prediction. AnyBURL [1] is the closest published counterpart of the implemented system: it samples paths bottom-up, generalises them to rules, estimates the standard confidence of each rule by sampling, and ranks candidates by the highest confidence of any predicting rule; its authors report that object-identity constraints remove quasi-redundant rules with overrated confidence. Evoda [2]† searches a general Datalog rule space with genetic logic programming under the open-world assumption and replaces the standard confidence by the PCA confidence. Omran et al. [3] define support and confidence over distinct entity pairs and learn open-path rules that can generate queries for entities missing from the graph.

The implemented system takes three ideas from this group: support and confidence counted over *distinct entity pairs* [3] rather than paths, the PCA confidence of Evoda [2], and the maximum aggregation of AnyBURL [1] (a candidate takes the best score of its supporting rules). It enumerates one- and two-atom rules exhaustively instead of sampling or genetic search, which is transparent and reproducible but excludes longer rules and rules with constants.

Table 2.2: Comparative review: rule-based knowledge graph completion and logical rule learning.

Paper (year)	Objective and method	Inference or scoring	Data and evaluation	Contribution and limitation	Relevance to this project
[1] Meilicke et al. (2024); journal	Scalable symbolic rule learning; bottom-up path sampling and generalisation	Maximum confidence of predicting rules (standard confidence, sampled)	FB15k, YAGO3-10, Wikidata5M, Freebase; MRR, Hits@k, time, energy	Higher prediction quality and lower run time than baselines on large data. *Stated:* maximum aggregation is chosen for scale although other aggregations perform slightly better	Closest counterpart: maximum aggregation adopted (G1, G5)
[2]† Wu et al. (2022); conference	Rule learning under the OWA with a general rule language; genetic search	PCA confidence; rule confidence assigned to predictions	YAGO, DBpedia, Wikidata for rule quality; WN18, YAGO3-10 for completion; Hits@k, MRR	Higher-quality rules and more precise predictions than compared miners. *Stated:* future work on constants and rule simplification	Source of the PCA confidence used in the miner (G1, G6)
[3] Omran et al. (2022); journal	Rules that generate queries for missing entities; open-path rules (OPRL)	Standard confidence and head coverage over entity pairs	Adaptations of Freebase, YAGO2, Wikidata, Poker; precision and recall of queries	Mines accurate rules from KGs with up to 8M facts. *Own:* open-path setting differs from closed two-atom rules	Pair-based support and head coverage adopted (G6)
This project	Exhaustive one- and two-atom rules (symmetry, inverse, subrelation, chain, reversed chain)	Max Wilson lower bound of PCA confidence	CoDEx-M; exact-set benchmark precision, recall, F1	160 frozen rules. *Limits:* no rules longer than two atoms, no constants; 6 hub bodies skipped	Implemented (Ch. 4)

2.3 Rule confidence and candidate-specific scoring

Grouping criterion: papers that change how the evidence for a candidate is scored or combined, instead of using one global confidence per rule. SLogic [4]† argues that a global rule weight ignores the query and computes query-dependent rule scores with a graph neural network over the subgraph of the head entity; on FB15k-237 it reports MRR 0.30 and Hits@1 21.9%, and its authors state training and subgraph overhead and chain-like rules only as limitations. Patil et al. [5]† address the combination of rules: a probabilistic circuit over rule sets, without independence assumptions, reaches the peak performance of the AnyBURL baseline with 70–96% fewer rules on eight datasets, which include FB15K-237 and CoDEx-S (the small CoDEx variant, not CoDEx-M). SLogic changes the score of a rule; the circuits paper changes how rules are combined.

The implemented system does neither in the score itself: a candidate's score is the maximum Wilson lower bound of its rules' PCA confidence, which shrinks the score of rules with little

evidence but remains global per rule. Ranking V2 introduces one candidate-specific quantity, the number of distinct training proofs (body matches), but only as a tie-break inside groups of equal score. The cap experiment shows that this tie-break alone raises validation matches at every cap (Section 5.5).

Table 2.3: Comparative review: rule confidence and candidate-specific scoring.

Paper (year)	Objective and method	Inference or scoring	Data and evaluation	Contribution and limitation	Relevance to this project
[4]† Le et al. (2025); preprint	Query-dependent rule scores from the head-entity subgraph	GNN-based weights over simple-path rules; coverage penalty	FB15k-237, WN18RR, YAGO3-10, NATIONS; filtered MRR, Hits@k	On FB15k-237 MRR 0.30, Hits@1 21.9%. *Stated:* overhead of positive–negative sample generation and fixed-size subgraphs; chain-like Horn clauses only	Motivates candidate-specific evidence (G1, G2)
[5]† Patil et al. (2025); preprint	Smaller, interpretable rule sets; probabilistic circuits over rule sets	Exact or lower-bound probability of a candidate	Eight datasets incl. FB15K-237 and CoDEx-S; filtered Hits@k, MRR	70–96% fewer rules reach peak baseline performance. *Own:* no dedicated limitations section	Motivates principled aggregation of rule evidence (G2, G5)
This project	Max Wilson lower bound; V2 tie-break by distinct body matches	Global per-rule score; evidence used only for ties	CoDEx-M; validation and test exact-set metrics	V2 adds 21 test matches at cap 100 with the same 13 FP. *Limit:* evidence not in the score; not calibrated	Partly implemented

2.4 Ontology-based reasoning and semantic constraints

Grouping criterion: papers in which ontology semantics, class hierarchies or logical schemas influence reasoning. Reasoning based on ontologies is one of the main groups of KG reasoning [33]. Zhang et al. [6] survey logics and embeddings and observe that few methods support general ontological schemas. OneSRPL [7] injects ontology semantics into subgraph reasoning and reports better results than its baselines on inductive subsets of WN18RR, FB15k-237 and NELL-995; TransC [8] models concepts as spheres and instances as vectors; and Balakrishnan et al. [9] use domain ontologies for healthcare data interoperability without addressing completion.

In the cited completion work the ontology is an input to a learned model. In this project the Phase 3 RDF/OWL dataset contains the training assertions in a named graph, declares entities as `owl:NamedIndividual` and relations as `owl:ObjectProperty`, and deliberately adds no domain, range, inverse, symmetry or class axioms, because CoDEx-M supplies no verified entity types or relation constraints (Section 4.5). The rule miner reads its training named graph, but no ontology semantics enter rule selection, candidate generation or scoring, so the ontology is not yet a semantic constraint on predictions.

Table 2.4: Comparative review: ontology-based reasoning and semantic constraints.

Paper (year)	Objective and method	Inference or scoring	Data and evaluation	Contribution and limitation	Relevance to this project
[6] Zhang et al. (2024); conference	Survey of integrating logics (rules, ontological schemas) with embeddings	Not applicable (survey)	Not applicable	Rule measures such as confidence and head coverage may mislead under incomplete, noisy KGs. *Stated:* few methods support general ontological schemas	Positions rules and schemas as the interpretable side (G3, G4, G6)
[7] Wang et al. (2026); journal	Inductive completion with ontology semantics in subgraph reasoning and prompts	Neural subgraph scoring with ontology features	Inductive subsets of WN18RR, FB15k-237, NELL-995	Better than baselines, particularly in few-shot settings. *Stated:* generalisation to unseen relations not systematically evaluated	Shows ontology semantics can enter KGC (G3)
[8] Lv et al. (2018); conference	Distinguish concepts and instances in embeddings	Geometric containment of instances in concept spheres	YAGO-based data; link prediction, triple classification	Captures transitivity of instanceOf and subClassOf. *Own:* requires explicit concept hierarchies, which CoDEx-M lacks	Contrast with the untyped CoDEx-M ontology (G3)
[9] Balakrishnan et al. (2025); conference	Semantic interoperability of multi-centre healthcare data with ontologies	Not applicable	Not reported in the abstract	Supports consistent interpretation across institutions. *Own:* not a link-prediction method	Why ontologies check semantic consistency (G3)
This project	RDF/OWL TriG with train, labels and provenance named graphs	Not used in the score	185,584 train + 51,303 label + 23 provenance triples	Lossless, verified representation. *Limit:* no semantic constraint applied	Built; ontology-aware scoring planned

2.5 Explainable knowledge graph inference

Grouping criterion: papers that make the justification of a prediction a design goal. Rule-based methods can trace a prediction to its rule [1, 12]. The explainable path reasoning framework of Yu et al. [10] mines paths, scores candidates with a BERT-based encoder and builds multi-hop explanations by beam search; its authors report that two- or three-hop paths are often sufficient and that interpretability is limited in sparse regions. Li et al. [11] discuss knowledge graphs as a basis for transparent, responsible AI. The implemented explanation stores, for every prediction, all supporting rules with their statistics and every body grounding (11,059 groundings for the

3,239 cap-100 predictions), and the prediction explorer draws them as a subgraph (Figure 4.7). It is of the same kind as a short path explanation but produced by counting, without a language model, and it has not been evaluated with users.

Table 2.5: Comparative review: explainable knowledge graph inference.

Paper (year)	**Objective and method**	**Inference or scoring**	**Data and evaluation**	**Contribution and limitation**	**Relevance to this project**
[10] Yu et al. (2026); journal	Explainable path-based completion	BERT path scoring; beam search	Not extracted	Competitive among explainable methods, narrower gap to black-box models. *Stated:* offline path mining gives a decoupled pipeline without end-to-end optimisation	Supports explaining a candidate by its supporting paths (G4, G5)
[11] Li et al. (2025); magazine	KGs for transparent, responsible AI	Not applicable	Not applicable	Research opportunities and challenges. *Own:* position paper	Motivation for auditable reasoning (G4)
This project	Rules, statistics and every grounding stored per prediction; explorer	Not applicable	Explanations exist by construction	Path-level explanations. *Limit:* not evaluated with users	Implemented

2.6 Neural-symbolic reasoning and differentiable rule learning

Grouping criterion: papers that learn rule structure or rule weights with gradient-based, probabilistic or neural methods. Neural LP [12]† learns rule structure and confidences jointly with a differentiable formulation built on TensorLog operators; on FB15KSelected (14,541 entities, 237 relations, 20,466 test triples) it reports MRR 0.24 and Hits@10 36.2%. RNNLogic [13] treats rules as latent variables optimised with an EM algorithm and shows that rules and embeddings are complementary; NCRL [14] learns compositional rules end to end and is reported to train about 100 times faster than other rule learners; logical neural networks [15] learn rules with embeddings (scalability is a stated limitation); and R-CQA [16] injects rules into complex query answering. These methods learn rule selection or weights and are evaluated by filtered ranking. The implemented system selects rules by thresholds on counts and reports exact-set metrics, so none of these results is comparable with this project's numbers.

Table 2.6: Comparative review: neural-symbolic reasoning and differentiable rule learning.

Paper (year)	Objective and method	Inference or scoring	Data and evaluation	Contribution and limitation	Relevance to this project
[12]† Yang et al. (2017); conference	Learn rule structure and confidences jointly (differentiable)	Sum of learned rule confidences; chain rules	UMLS, Kinship, WN18, FB15K, FB15KSelected, WikiMovies; filtered MRR, Hits@10	FB15KSelected: MRR 0.24, Hits@10 36.2%. *Own:* no limitations section	Rule confidence as an explicit, interpretable quantity; short rules suffice (G1, G4)
[13] Qu et al. (2021); conference	Probabilistic rule learning; rules as latent variables (EM)	Reasoning predictor weights rules	Four datasets incl. FB15k-237; MRR, Hits@k	Effective with few rules per relation. *Stated:* the reasoning predictor is simple; left to future work	Rule selection by learning rather than thresholds (G1, G5)
[14] Cheng et al. (2023); conference	Scalable end-to-end compositional rule learning	Learned scores of composed rules	Large KGs incl. FB15k-237; MRR, Hits@k, training time	About 100 times faster training than rule learners; systematic generalisation. *Own:* limitations not found in the extracted text	Benchmark for rule-learner scalability (G5, G7)
[15] Sen et al. (2022); conference	Logical neural networks with embeddings and rules	Learned rule weights	English benchmarks; MRR, Hits@k	About 10% relative gain over rule-based methods, a further 7.5% with embeddings. *Stated:* larger KGs need more GPU resources	Rule–embedding combination keeps rules interpretable (G5)
[16] Zhang et al. (2025); conference	Inject rules into complex query answering	Embedding query scores with rule injection	Not extracted	Rule injection improves base models. *Own:* targets complex queries, not single edges	Rules carry useful background knowledge (G3)
This project	Thresholds on support, PCA confidence, head coverage and lift	Counted statistics	Exact-set metrics only	Transparent and cheap. *Limit:* no learned rule selection; not comparable	Not implemented

2.7 Knowledge graph embeddings and link prediction

Grouping criterion: papers whose principal method is a vector-space scoring function for triples. Embeddings are the main alternative to rule-based completion and the natural baselines for later comparison; they rank all entities but give no supporting facts for an individual prediction. The families differ in how a relation acts on entities: translations [20–23], bilinear or complex factorisation [17, 18], and rotations [19]. DistMult also shows that Horn rules can be mined from learned embeddings [17], and RotatE's relation composition corresponds to the two-atom

chain rules mined here [19]; its symmetry and inversion patterns correspond to the symmetry and inverse rules (for example spouse and sibling) that produce most of this project's test matches. No embedding model is implemented in this project. Table 2.7, at the end of Section 2.9, compares these approaches with the proposed one.

2.8 Graph neural networks and advanced learning methods

Grouping criterion: papers that learn from graph neighbourhoods, paths or subgraphs. GNN completion models aggregate neighbour, edge or text information [25–27] or model query-related paths and subgraphs at several granularities [24], and the reviews [28,29] place them among graph learning methods; [30–32] apply neural reasoning to multi-hop queries and industrial settings. They can use local evidence, which a global rule score lacks, but need training and are less transparent; they are not implemented here.

2.9 Knowledge graph surveys, benchmarks and evaluation

Grouping criterion: surveys, reviews and papers that define the benchmarks, protocols and assumptions used to assess completion. Liu et al. [33] group reasoning into ontology-based, rule-based, neural-rule and hybrid methods, Niu et al. [34] state that reasoning follows the open-world assumption, and Ji et al. [36] report studies of probability calibration. Li et al. [35] list CoDEx, with its small, medium and large variants, among Wikidata-derived completion benchmarks. The embedding surveys classify models by information used, representation space, structure and literals [37–39,41,42]; the selected survey of Cao et al. [39]† organises models by algebraic, geometric and analytical representation spaces and collates results from downstream tasks, which is the basis for choosing embedding baselines later. Biswas et al. [40] note that most techniques are assessed on human-curated link-prediction benchmarks. Ranking-based papers report filtered MRR and Hits@k [4,12]; SLogic also handles tied scores, because a naive ranking of a rule-based system can assign a good rank when no rule applies [4]. Zhang et al. question the silver-standard correctness of benchmark evaluation [6]. These points justify the protocol of this project: exact-set metrics with unlabelled predictions treated as unknown, no MRR or Hits@K until a valid ranking protocol exists, and a documented separation of validation and test use.

Table 2.7: Comparative review: embeddings, graph neural networks, surveys, benchmarks and evaluation.

Papers	Method and contribution	Data and evaluation	Limitation	Relevance to this project
[17,18]	Bilinear (diagonal) and complex-valued factorisation; DistMult reports top-10 accuracy 73.2% against 54.7% for TransE on Freebase and mines Horn rules from embeddings; ComplEx handles symmetric and antisymmetric relations	Freebase; standard link-prediction benchmarks	*Own:* no supporting facts for one prediction	Baseline family; rules can be mined from embeddings (G1)

Papers	Method and contribution	Data and evaluation	Limitation	Relevance to this project
[19]	Relations as rotations in complex space; symmetry, antisymmetry, inversion and composition; self-adversarial negative sampling	Link-prediction benchmarks	*Own:* composition is geometric, not explicit	Composition and symmetry correspond to mined rules; baseline
[20–23]	Translations of entity embeddings (TransE), refined with hyperplanes, relation-specific spaces or relational mapping properties	Link prediction; triple classification	1-to-N, N-to-1 and N-to-N relations are hard for TransE	Baseline family; mapping properties relate to candidate explosion (G5)
[24–27]	GNN completion: multi-granularity paths and subgraphs (MulGA), edge-aware aggregation (ERA-GCN), embedding-based GCN, text semantics (SEGCN)	FB15k-237, WN18RR and others; MRR, Hits@k	*Own:* neural, less transparent; abstract-level reading	Local evidence (G2); later baselines
[28, 29]	Surveys of GNNs for KGs and of GNN architectures	Not applicable	*Own:* general background	Context for neural alternatives
[30–32]	Multi-hop logical queries on multi-modal KGs; process KG reasoning; industrial fault prediction	Application datasets	*Own:* domain-specific	Application motivation
[33–36]	Surveys of reasoning and completion: ontology, rule, neural and hybrid groups; OWA; calibration; CoDEx listed as a benchmark	Not applicable	*Own:* surveys, no new method	Context; OWA; calibration (G6)
[37–39]†, [40–42]	Surveys of embedding techniques by information, representation space, structure and literals; benchmark critique	Not applicable	*Own:* embedding-focused	Baseline selection; evaluation limits (G6)
[43]	Temporal reasoning: representation learning versus transparent symbolic rules	Not applicable	*Own:* temporal setting	Interpretability trade-off
This project	Embeddings and GNNs not implemented; exact-set benchmark precision, recall, F1 and positive hit rate; unlabelled predictions unknown	CoDEx-M splits with supplied negatives	No MRR or Hits@K; no baseline; test split used twice	Baselines and ranking protocol planned

2.10 Development of the field

Table 2.8 orders the main reviewed approaches by publication year. It shows how the field moved from vector-space scoring to explicit rules and, more recently, to combinations of rules, neural models and ontology semantics. The order of publication is not the order of relevance used in the rest of the chapter.

Table 2.8: Development of the main reviewed approaches by year.

Year	Approach	Main idea
2013–2016	Translation and factorisation embeddings [17, 18, 20–23]	Relations as translations, projections or factorisations of entity vectors; scalable but without supporting facts
2017	Neural LP [12]†	Differentiable learning of rule structure and confidence
2018	TransC [8]	Concepts as spheres and instances as vectors
2019	RotatE [19]	Relations as rotations; symmetry, inversion and composition
2021	RNNLogic [13]	Rules as latent variables learned with EM
2022	Evoda [2]†, logical neural networks [15]	Genetic rule search with PCA confidence; neural learning of rules
2023	NCRL [14]	Compositional rules learned end to end
2024	AnyBURL [1]; Cao et al. survey [39]†	Scalable bottom-up rule learning; representation-space view of embeddings
2025	SLogic [4]†, probabilistic circuits [5]†	Query-dependent rule scores; probabilistic combination of rules
2026	OneSRPL [7], EPR [10]	Ontology semantics inside neural reasoning; explainable path reasoning

2.11 Key lessons from the literature

Four lessons shape the design of this project. First, explicit rules give explanations that embeddings do not, and compact rule sets can match larger ones [1, 5], so a small, transparent rule language is a defensible starting point. Second, how a rule is scored matters as much as which rules are found: the standard confidence is questioned for incomplete graphs [2, 6], and query-dependent scores improve results at the cost of training [4]. Third, ontology semantics help when they are integrated into the model, but they have so far been used mainly inside neural systems [7], which leaves a simple, count-based use open. Fourth, evaluation under an incomplete graph needs care: ranked metrics need a defined candidate universe and a tie policy [4], and a non-match is unknown, not false [34].

2.12 Research gap analysis

Table 2.9 lists the gaps that the literature supports. For each it names the papers that address it, what remains open, and the status in this project: *addressed* means implemented and measured in this report, *partly* means implemented in a limited form, and *planned* means not implemented.

Table 2.9: Research gap analysis: gaps supported by the literature and their status in this project.

ID	Gap	Support	Addressed in the literature; what remains	This project	Status
G1	Global rule confidence	[1, 2, 4, 6]	Query-dependent scores and PCA confidence exist; standard confidence is questioned under incomplete data	PCA confidence with a Wilson lower bound per rule; one value per rule; not calibrated	Partly
G2	Candidate-specific evidence	[4, 5]	Subgraph evidence and probabilistic combination exist; costly or neural	Distinct body matches computed for all 184,487 candidates but used only as V2 tie-break	Partly
G3	Ontology-aware scoring	[6–8]	Few methods support ontological schemas; integration is inside neural models	RDF/OWL dataset generated; no domain/range or class axioms; not used in mining or scoring	Planned
G4	Explainability	[4, 6, 10, 12]	Rules and paths explain predictions; explanations rarely evaluated	All rules and groundings per prediction; explorer; no user evaluation	Addressed (path level)
G5	Scalability and candidate volume	[1,4,5,14]	Fewer rules, coverage penalties and scalable learners	6 hub bodies over 5M paths skipped; per-relation caps 100–1,000 compared on validation	Partly
G6	Evaluation in an incomplete graph	[2,4,6,34]	PCA confidence, OWA, tie handling, benchmark critique	Unlabelled predictions unknown; benchmark precision over labelled predictions; no MRR/Hits@K; test used twice	Partly
G7	Rule coverage	[1, 10, 14]	Longer rules and paths raise coverage	One- and two-atom rules only; 11 test relations have no prediction; recall ceiling 71.97% at cap 1,000	Partly

Synthesis. Rule discovery and confidence estimation are well studied but mostly global: AnyBURL, Evoda and Neural LP assign each rule one value [1, 2, 12]; SLogic and the circuits paper make evidence candidate-specific or combine it jointly at the cost of training [4, 5]; and ontology semantics enter completion only inside neural models [7]. What remains open is a transparent, count-based candidate score that combines rule reliability with candidate-specific evidence and a semantic-consistency check, evaluated under the open-world assumption with a valid ranking protocol. The project addresses part of this: it implements an explainable rule pipeline with PCA confidence, Wilson bounds and full groundings (G4), measures the effect of an evidence tie-break (G2) and of per-relation caps (G5) under an open-world protocol (G6), and builds the ontology. Ontology-aware scoring, evidence inside the score, longer rules, ranked evaluation and baseline comparison are proposed future work, and the project does not claim novelty or superiority over the reviewed methods.

3 Project Overview, Problem Statement and Objectives

3.1 Problem statement

Given an incomplete knowledge graph G (here the CoDEx-M training split), the task is to propose missing triples that are absent from G, to give each proposal a score that reflects the reliability of the rules that support it, and to explain each proposal by the training facts that produce it. A rule $B(x, z) \Rightarrow h(x, z)$ with a body of one or two atoms is mined from G with its support, standard confidence, PCA confidence, head coverage and Wilson lower bound; applying the rule to G yields candidates $h(x, z) \notin G$. Each candidate is ranked by the best Wilson lower bound of its rules, ties are broken by a documented order (V1 or V2), a per-relation cap limits the number kept, and the kept predictions are evaluated against held-out positives and supplied negatives under the open-world assumption.

The limitations that define the scope of the problem are verified in Chapter 5: candidate explosion around hub entities (six body shapes exceed five million paths; 184,487 unique candidates for 33 relations), ties that make the cap decide by entity ID at cap 100 (31 of 32 capped relations in V1), a global per-rule score, limited coverage (11 relations with test positives receive no prediction), open-world uncertainty (most predictions are unlabelled), and an ontology that is not yet used in scoring.

3.2 Objectives

The project has seven objectives for Phase 1. Each has a planned extension that is future work and is not claimed as done (Table 3.1).

Table 3.1: Objectives, status, evidence and planned extensions.

ID	Objective	Phase 1	Evidence (repository)	Future extension (not done)
O1	Inspect and verify the CoDEx-M splits	Completed	`outputs/phase1_dataset_report.md`: counts, format, zero overlap	–
O2	Construct a relation-labelled training graph	Completed	17,050 nodes, 185,584 edges; GraphML, statistics, viewer	–
O3	Generate an RDF/OWL ontology representation	Completed; not used in scoring	TriG with 3 named graphs; reload and allowlist checks passed	Verified schema; ontology-aware scoring with ablation
O4	Mine and review logical rules	Completed	194 selected rules (run_grid_ext); 160 frozen after validation review	Longer rules, constants, object identity
O5	Generate, score and rank candidate edges	Completed	184,487 candidates; V1 and V2 rankings; caps 100–1,000	Evidence inside the score; calibration

ID	Objective	Phase 1	Evidence (repository)	Future extension (not done)
O6	Explain and explore predictions	Completed; not user-tested	Prediction explorer with provenance; cap-comparison dashboard	User evaluation
O7	Evaluate with a documented protocol	Completed	Phase 6 test evaluation; validation-based cap selection; final test	Ranking protocol (MRR, Hits@K); baselines

3.3 Original proposal and its realisation

The project abstract proposed a pipeline in which an ontology is built from the knowledge graph, transformed into logical rules, applied by an inference engine to derive candidate missing edges, and scored by a confidence that combines the reliability of the rule, supporting evidence, the strength of relationship paths and consistency with the ontology; candidates would be ranked and evaluated with precision, recall, F1, MRR and Hits@K. Table 3.2 compares each element with what is implemented, so that the report does not claim more than the code does.

Table 3.2: Elements of the original proposal compared with the implementation.

Element of the proposal	Status	What the implementation does
Ontology built from the knowledge graph	Implemented	RDF/OWL TriG dataset: training assertions, labels and provenance in separate named graphs
Ontology transformed into logical rules	Not as proposed	Rules are mined from the training named graph's triples; no ontology axiom is used to create them
Inference engine derives candidate edges	Implemented	One round of 160 frozen rules over training facts; training triples and duplicates removed
Confidence from rule reliability	Implemented	PCA confidence and its Wilson lower bound per rule; best rule gives the score
Confidence from supporting evidence	Partly	Distinct body matches counted for every candidate; used as V2 tie-break only
Confidence from path strength	Planned	Not implemented
Confidence from ontology consistency	Planned	Not implemented
Ranking of candidates	Implemented	Per-relation ranks (V1, V2) and caps 100, 250, 500, 1,000
Precision, recall, F1	Implemented	Exact-set benchmark metrics on validation and test, plus positive hit rate
MRR and Hits@K	Planned	Requires a per-query ranking protocol with a defined candidate universe

3.4 Scope and expected contribution

The scope of Phase 1 is a complete, reproducible rule-based completion baseline on one benchmark, with an evaluation protocol that keeps the test split for final checks and treats unlabelled predictions as unknown. The expected contribution is (i) a thematic, comparative review of 43 papers with a research-gap analysis; (ii) an explainable pipeline whose every prediction carries

its rules and training groundings; (iii) a measured comparison of two tie-break policies and four cap sizes, selected on validation with a rule frozen in advance; and (iv) documented limitations that define Phase 2: ontology-aware and evidence-aware scoring, ranked evaluation and baseline comparison. Neural or embedding baselines, user studies and ontology semantics in the score are outside the Phase 1 scope.

3.5 Research questions

Three questions guide the evaluation; Chapter 5 answers them with the verified results.

- **RQ1. How much of the held-out data can one- and two-atom rules recover under a per-relation cap?** Section 5.6.
- **RQ2. Does candidate-specific evidence, used as a tie-break, improve the ranking?** Sections 5.4 and 5.5.
- **RQ3. What is the trade-off when the cap grows, and what limits the approach?** Sections 5.5 and 5.7.

3.6 Comparative analysis

Chapter 2 compares the approach with the published ones theme by theme (Tables 2.2–2.7); the last row of each table summarises the position of this project. Published results on FB15k-237-type data use filtered ranking over all entities: Neural LP reports MRR 0.24 and Hits@10 36.2% on FB15KSelected [12], and SLogic reports MRR 0.30 and Hits@1 21.9% on FB15k-237 [4]. This project reports exact-set metrics on CoDEx-M against supplied positives and negatives. The two kinds of number differ in dataset, candidate universe, negatives and metric, so they are *not comparable*, and superiority is not claimed.

4 Proposed and Implemented Methodology

4.1 Published approaches and the conceptual pipeline

Rule-based missing-edge prediction in the literature follows a common conceptual pipeline: rule discovery, rule confidence estimation, candidate generation, candidate scoring, optional ontology integration, explanation generation and evaluation. Table 4.1 shows how the most relevant published approaches realise each stage and where the implemented system stands. The rows for published methods are literature-derived; only the last row describes code that was run in this project.

Table 4.1: Conceptual pipeline stages in published approaches and in the implemented system.

Approach	Rule discovery	Confidence	Candidate generation	Candidate scoring	Ontology	Explanation	Evaluation
AnyBURL [1]	Path sampling, generalisation	Standard, sampled	Rule application	Maximum confidence	–	Rule per prediction	Filtered MRR, Hits@k
Evoda [2]†	Genetic search	PCA confidence	Rule application	Rule confidence	–	Rule	Hits@k, MRR
SLogic [4]†	Simple paths	GNN, query-dependent	Rule application	Learned	–	Rule	Filtered MRR, Hits@k
Circuits [5]†	AnyBURL rules	Circuit probability	Rule application	Joint probability	–	Rule set	Filtered Hits@k, MRR
Neural LP [12]†	Differentiable	Learned	Operator chains	Sum of confidences	–	Rule	Filtered MRR, Hits@10
OneSRPL [7]	–	–	Subgraphs	Neural	Ontology features	–	Inductive ranking
EPR [10]	Path mining	–	Paths	BERT encoder	–	Multi-hop path	Ranking
This project	Exhaustive 1–2 atom rules	PCA conf. + Wilson bound	Rule application, one round	Max Wilson; V1/V2 ties; caps	Built, not in score	All groundings	Exact-set; OWA labels

4.2 Architecture of the implemented system

Figure 4.1 shows the architecture of the implemented system, drawn from the source code and the saved manifests. Training data flow into graph construction, ontology generation, rule mining and candidate generation; no validation or test file is opened by the miner (`phase4_rule_mining.py`) or the predictors (`phase5_predict.py`, `phase5_rank_v2.py`), which their tests check by recording every opened file. The validation split is used in two places only: the review of rule thresholds and exclusions in Phase 4, and the selection of the cap and ranking version. The test split enters only the evaluation stage, and it was used twice: first for the cap-100 V1 and V2 baseline (Phase 6), and later for the single configuration selected on validation. The ontology is shown as a separate generated representation: the miner reads its training named graph as a source of triples, but no ontology semantics influence the score. The editable source of the diagram is `KG_CoDExM_Project_Architecture.dot`.

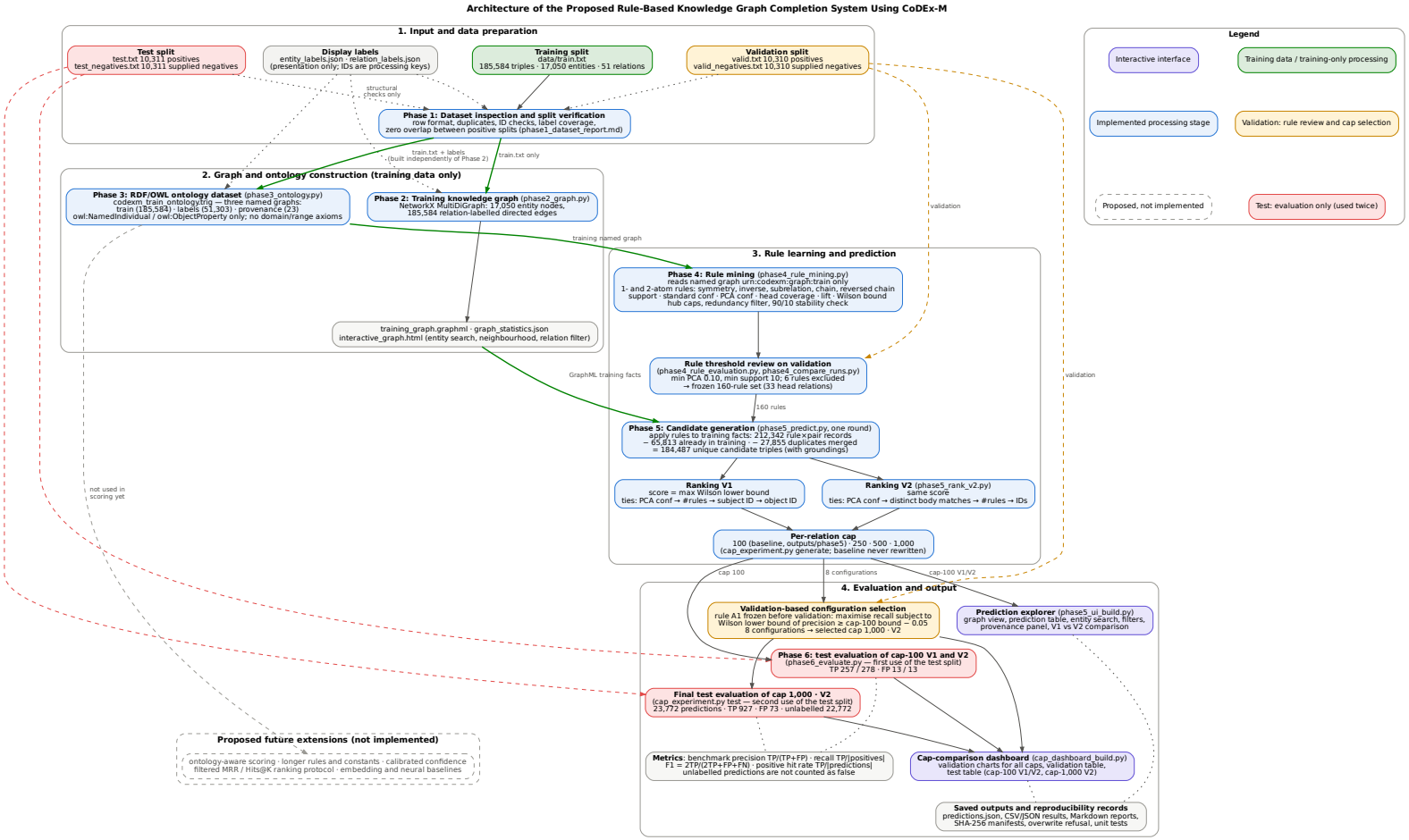


Figure 4.1: Architecture of the Proposed Rule-Based Knowledge Graph Completion System Using CoDEx-M. Green arrows: training-only data flow; amber dashed arrows: validation use (rule review, cap selection); red dashed arrows: test use (cap-100 baseline first, selected cap-1,000 V2 configuration second); dashed grey box: proposed extensions that are not implemented. The ontology does not participate in scoring.

4.3 Data and splits

CoDEx-M [44] is a Wikidata-derived completion benchmark whose entities and relations keep their Wikidata identifiers (Q..., P...). The five local files were inspected in Phase 1 (Table 4.2). Every row parsed as three tab-separated IDs, there were no duplicates, malformed rows or unmapped IDs, and no positive triple is shared between any two positive splits. The supplied negative files have as many rows as their positive splits, contain no positive triple, and were checked structurally only; their semantic validity was not assessed. The label files `entity_labels.json` and `relation_labels.json` are used for display only.

File	Triples	Entities	Relations	Use in the pipeline
`train.txt`	185,584	17,050	51	Graph, ontology, rules, candidates
`valid.txt`	10,310	9,124	45	Rule review; cap selection
`valid_negatives.txt`	10,310	4,990	43	Rule review; cap selection
`test.txt`	10,311	9,137	43	Final evaluation only
`test_negatives.txt`	10,311	4,972	43	Final evaluation only

Table 4.2: CoDEx-M files, verified from `outputs/phase1_dataset_report.md`. Entities and relations are those occurring in each file.

4.4 Graph construction and visualisation

`phase2_graph.py` reads `train.txt` only and builds a NetworkX `MultiDiGraph` in which each unique triple is a directed edge keyed by its relation ID, so different relations between the same entities are kept. The graph has 17,050 nodes and 185,584 edges; it is exported as `training_graph.graphml`, together with `graph_statistics.json` (relation frequencies, two-hop path observations), `filtering_decisions.json` (source hash, filtering policy; no rows were filtered) and an offline interactive viewer. The relation distribution is highly skewed (Figure 4.2): occupation (P106) alone holds 64,345 training triples, while eight relations have fewer than 20.

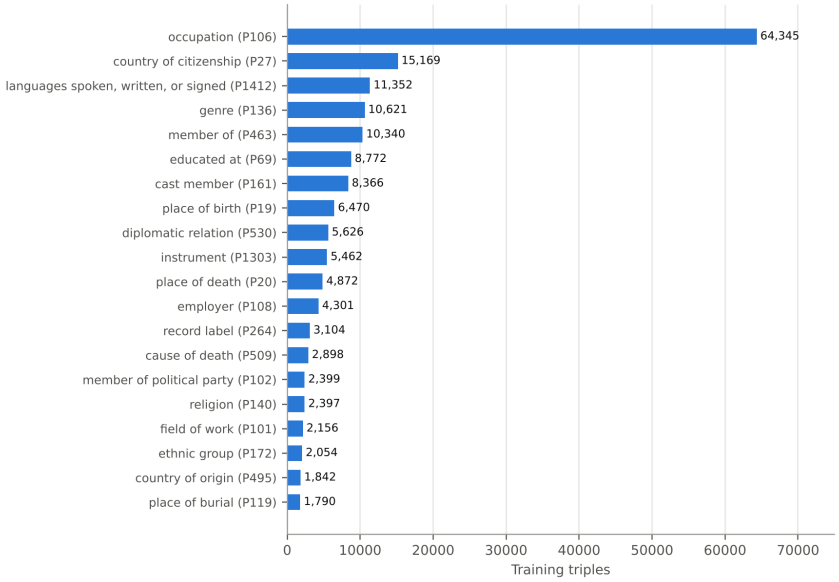


Figure 4.2: The 20 most frequent relations in the CoDEx-M training graph (`graph_statistics.json`).

The interactive viewer (`outputs/phase2/interactive_graph.html`) bundles the complete training graph and draws only a bounded neighbourhood or relation subgraph at a time. Figure 4.3 shows the one-hop neighbourhood of Katherine Jackson (Q238402): 15 training triples, one incoming (spouse, from Joe Jackson) and fourteen outgoing (nine child, two occupation, one each for language, ethnic group and citizenship). This entity is used again in the worked example of Section 4.8.

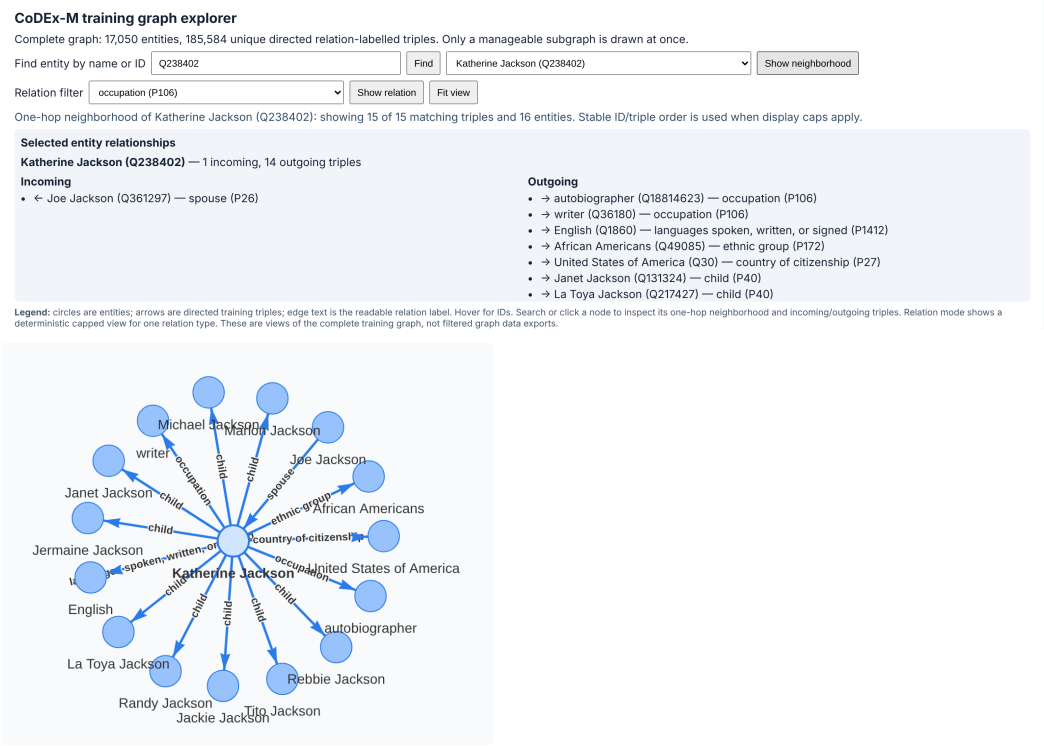


Figure 4.3: Screenshots of the Phase 2 training-graph explorer: entity search and relationship panel (top) and the drawn one-hop neighbourhood of Katherine Jackson (bottom). Only training triples are shown.

4.5 RDF/OWL ontology generation

`phase3_ontology.py` converts `train.txt` and the two label maps into an RDF dataset in TriG (`outputs/phase3/codexm_train_ontology.trig`, 20.5 MB). It uses project URNs (`urn:codexm:entity:Q...`, `urn:codexm:relation:P...`) and three named graphs:

- `urn:codexm:graph:train` – exactly the 185,584 training triples, with no labels or types;
- `urn:codexm:graph:labels` – 51,303 triples: `rdfs:label`@en, a `sourceId` literal, and the generic declarations `owl:NamedIndividual` (entities) and `owl:ObjectProperty` (relations);
- `urn:codexm:graph:provenance` – 23 triples: source path and SHA-256, counts and converter version.

The design document (`ontology_design.md`) records why no domain, range, inverse, symmetry, transitivity, property-chain or class axioms are declared: the local data provide no verified entity types or relation constraints, and a label that sounds symmetric is a hypothesis, not evidence. The builder reloads its output and verifies an exact set match with the source, reversible ID mapping, the axiom allowlist and a deterministic rebuild (`phase3_manifest.json`). Listing 4.1 is a genuine excerpt, and Figure 4.4 visualises the same triples.

Listing 4.1: Excerpt of `codexm_train_ontology.trig` (lines 1–3, 68506–68508, 105648, 204412–204414, 236818–236820 and part of the provenance graph).

```
# CoDEx-M Phase 3 RDF dataset generated by phase3_ontology.py 3B.1.0.
# Canonical TriG subset: full IRIs, one triple per line, lines sorted within each graph.
# <urn:codexm:graph:train> holds only asserted training triples; labels and provenance are separate.
<urn:codexm:graph:train> {
<urn:codexm:entity:Q238402> <urn:codexm:relation:P27> <urn:codexm:entity:Q30> .
<urn:codexm:entity:Q238402> <urn:codexm:relation:P40> <urn:codexm:entity:Q131324> .
<urn:codexm:entity:Q238402> <urn:codexm:relation:P40> <urn:codexm:entity:Q217427> .
<urn:codexm:entity:Q361297> <urn:codexm:relation:P26> <urn:codexm:entity:Q238402> .
}
<urn:codexm:graph:labels> {
<urn:codexm:entity:Q238402> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/2002/07/
    owl#NamedIndividual> .
<urn:codexm:entity:Q238402> <http://www.w3.org/2000/01/rdf-schema#label> "Katherine Jackson"@en .
<urn:codexm:entity:Q238402> <urn:codexm:vocab:sourceId> "Q238402" .
<urn:codexm:relation:P26> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/2002/07/owl#
    ObjectProperty> .
<urn:codexm:relation:P26> <http://www.w3.org/2000/01/rdf-schema#label> "spouse"@en .
<urn:codexm:relation:P26> <urn:codexm:vocab:sourceId> "P26" .
}
<urn:codexm:graph:provenance> {
<urn:codexm:dataset:train> <urn:codexm:vocab:sourceFile> "data/train.txt" .
<urn:codexm:dataset:train> <urn:codexm:vocab:uniqueTripleCount> "185584"^^<http://www.w3.org/2001/XMLSchema
    #integer> .
<urn:codexm:dataset:train> <urn:codexm:vocab:note> "Asserted CoDEx-M training facts only. No inferred,
    predicted, validation, test or negative triples; no domain, range, inverse, symmetry or other semantic
     axioms." .
}
```

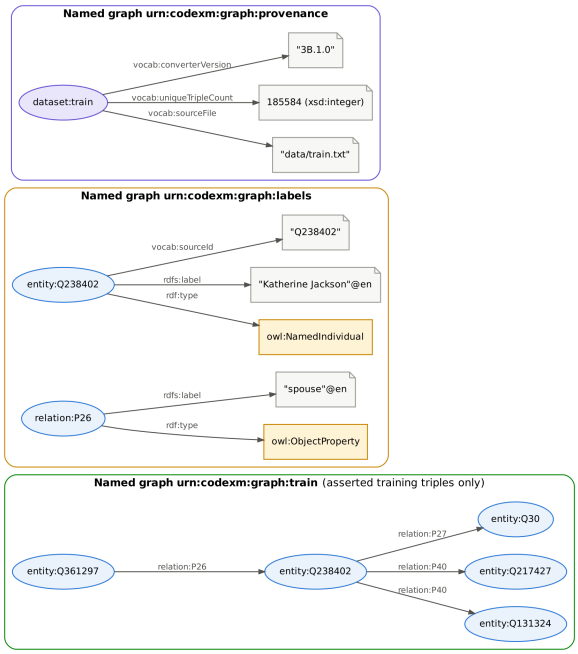


Figure 4.4: Visualisation of the triples of Listing 4.1 by named graph. The graphs are kept apart so that a consumer selects the training assertions explicitly.

4.6 Rule mining and validation review

`phase4_rule_mining.py` checks the TriG hash against the Phase 3 manifest, parses it and uses only the named graph `urn:codexm:graph:train`. It enumerates five rule shapes over relation IDs

(one-atom symmetry $p(z,x) \Rightarrow p(x,z)$, inverse $p(z,x) \Rightarrow q(x,z)$ and subrelation $p(x,z) \Rightarrow q(x,z)$; two-atom chains $p_1(x,y) \wedge p_2(y,z) \Rightarrow h(x,z)$ and chains with reversed atoms), for the 43 head relations with at least 20 training triples. Of 10,506 body shapes, 7,296 do not join and 3,204 are evaluated; six bodies through hub entities exceed the limit of five million paths and are skipped (for example $\mathrm{P106}(x,y) \wedge \mathrm{P106}(z,y)$, "shares an occupation", with 96,828,205 paths).

For a rule $R : B(x,z) \Rightarrow h(x,z)$ over the training graph G, counts are taken over distinct pairs:

$$\mathrm{supp}(R) = |\{(x,z) \in \mathrm{Body}(R) : h(x,z) \in G\}|, \qquad \mathrm{conf}(R) = \frac{\mathrm{supp}(R)}{|\mathrm{Body}(R)|}, \tag{4.1}$$

$$\mathrm{pca}(R) = \frac{\mathrm{supp}(R)}{|\{(x,z) \in \mathrm{Body}(R) : \exists z'.\, h(x,z') \in G\}|}, \qquad \mathrm{hc}(R) = \frac{\mathrm{supp}(R)}{|\{(x,z) : h(x,z) \in G\}|}, \tag{4.2}$$

where the PCA side (subject or object) is chosen per head relation. With $p = \mathrm{pca}(R)$, n the PCA body size and $z = 1.96$, the Wilson lower bound is

$$W(R) = \frac{p + \frac{z^2}{2n} - z\sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}. \tag{4.3}$$

Recomputing Equation (4.3) from the saved counts reproduces the stored values (for example 335/354 gives 0.9177 for spouse symmetry). For swap-invariant rules (symmetry and shared-subject or shared-object bodies) one unordered couple $\{x,z\}$ is one counting unit, so a pair and its reverse are not counted twice. Candidates must pass minimum support, PCA-body size, head coverage, PCA confidence and lift (confidence over the head prior), and redundant or dominated rules are removed; a 90/10 split of the training data checks stability.

Two runs exist and are preserved: `run_default` (166 rules) and `run_grid_ext` (194 rules, couple counting). `phase4_rule_evaluation.py` scored the frozen rules on the validation split over a grid of PCA and support thresholds; the floor pca ≥ 0.10, supp ≥ 10 was kept because looser points raised validation F1 by less than the adoption bar of 0.01. Six rules were excluded because they were unstable, depended on one middle entity for all support, or used a body relation with fewer than 20 triples. The result is the frozen list `phase5_candidate_rules_proposed.json`: **160 rules** for 33 head relations (67 chains, 70 reversed chains, 17 subrelations, 4 symmetry and 2 inverse rules; 8 use couple counting). Table 4.3 lists genuine rules from this set.

Table 4.3: Genuine rules from the frozen 160-rule set (`run_grid_ext/selected_rules.json`). Supp. = support; Body = body size; Std = standard confidence. Wilson bounds in the set range from 0.079 to 0.957.

Rule (labels)	Supp.	Body	Std	PCA	Wilson
child(y,x) ∧ child(y,z) ⇒ sibling(x,z) [couples]	127	128	0.992	0.992	0.957
spouse(z,x) ⇒ spouse(x,z) [couples]	335	437	0.767	0.946	0.918
child(x,y) ∧ child(z,y) ⇒ spouse(x,z) [couples]	36	39	0.923	1.000	0.904
educated at(x,y) ∧ country(y,z) ⇒ country of citizenship(x,z)	543	695	0.781	0.841	0.810
place of death(x,y) ∧ official language(y,z) ⇒ languages spoken(x,z)	643	960	0.670	0.812	0.783
diplomatic relation(z,x) ⇒ diplomatic relation(x,z) [couples]	2,472	3,154	0.784	0.784	0.770

location of formation(y,x) ∧ country of origin(y,z) ⇒ country(x,z)	24	30	0.800	0.923	0.759
country of citizenship(x,y) ∧ official language(y,z) ⇒ languages spoken(x,z)	7,887	17,201	0.459	0.653	0.644

4.7 Candidate generation, scoring and V1/V2 ranking

`phase5_predict.py` reads the Phase 2 GraphML (its stored source hash must equal the hash of `train.txt`) and applies the 160 rules for *one* round: predictions are never reused as evidence. For every body pair (x, z) with $x \neq z$ it records the rule and all body groundings. Of 212,342 rule×pair records, 65,813 are skipped because the head is already a training triple, and 27,855 duplicates are merged, leaving **184,487 unique candidate triples**. A candidate t supported by rules $\mathcal{R}(t)$ receives

$$\text{score}(t) = \max_{R \in \mathcal{R}(t)} W(R). \tag{4.4}$$

Many candidates share a score because one rule produces them all, so the tie-break order decides which are kept under a cap:

- **V1**: score ↓, max PCA confidence ↓, number of supporting rules ↓, subject ID ↑, object ID ↑;
- **V2** (`phase5_rank_v2.py`): score ↓, max PCA confidence ↓, *distinct body matches* ↓, supporting rules ↓, subject ID ↑, object ID ↑.

A body match is one set of training triples that makes a supporting rule's body true for (x, z); matches are deduplicated across rules, recomputed from the training graph for all candidates, and cross-checked against the stored groundings. V2 does not change candidates, scores or provenance; it only reorders equal-score groups. At cap 100, V1 and V2 share 2,830 of their 3,239 predictions and each has 409 that the other lacks.

4.8 Worked examples from the saved results

A recovered test triple. The V2 prediction ranked first for spouse (P26) is (Katherine Jackson, spouse, Joe Jackson). Two rules support it (Figure 4.7): spouse symmetry, grounded by the training triple (Joe Jackson, spouse, Katherine Jackson), and the shared-child rule child(x, y) ∧ child(z, y) ⇒ spouse(x, z), grounded by nine shared children. Its score is the higher Wilson bound, 0.918, and it has 10 distinct body matches. The triple is a test positive.

A hub-driven error. The V2 prediction ranked first for country (P17) is (London, country, United States of America), score 0.759, from location of formation(y, x) ∧ country of origin(y, z) ⇒ country(x, z): a band formed in London whose country of origin is the USA. The prediction is unlabelled and plausibly false; it shows that a high rule statistic does not establish a fact, and that more proof paths (89 matches here) often come from hub entities.

4.9 Prediction explorer

`phase5_ui_build.py` builds a static web interface (`outputs/prediction_ui/`, 16 MB) from the V1 and V2 prediction files, the training graph and the label maps; it never opens validation or test files and refuses to overwrite an existing build. It loads all 3,648 unique predicted triples (6,478 version rows) and loads provenance and training edges per relation and per entity on demand. Its

four tabs are a graph view, a sortable prediction table, a V1-versus-V2 comparison and a read-only view of the Phase 6 results. Filters for version, relation, score range, top-N per relation, text and entity apply to every tab. Figures 4.5–4.8 are screenshots of the working interface captured from the project files in a headless Chromium browser.

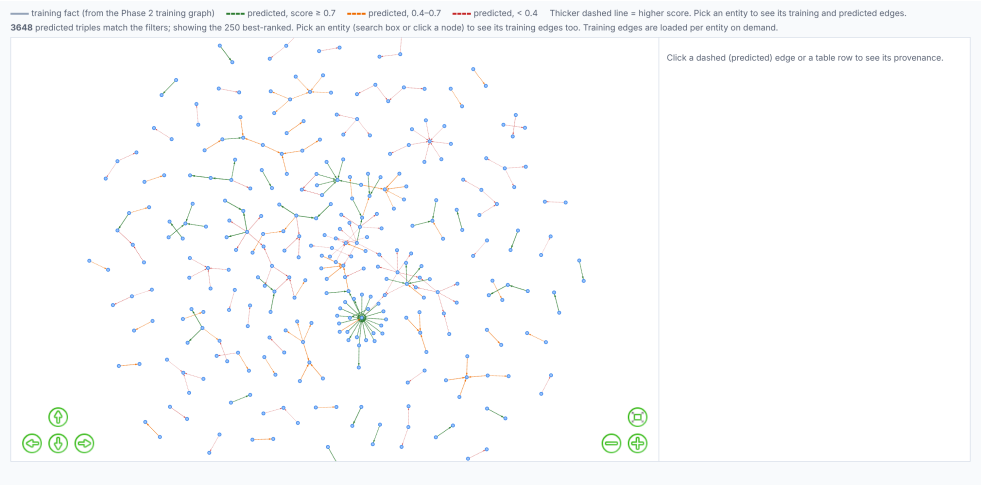


Figure 4.5: Prediction explorer, default graph view: the 250 best-ranked of 3,648 predicted triples; dashed edges are predictions coloured by score band (green ≥ 0.7, orange 0.4–0.7, red < 0.4).



Figure 4.6: Entity search and entity view for Katherine Jackson: 15 training edges (solid) and 2 predicted edges (dashed): spouse → Joe Jackson (score 0.918) and place of burial → Forest Lawn Memorial Park (score 0.377).

4.10 Evaluation protocol and metrics

`phase6_evaluate.py` labels each prediction once: *positive* if it is in the positive file of the split, *negative* if it is in the supplied negative file, and *unlabelled* otherwise. Unlabelled predictions are not treated as false; for information only, test-unlabelled predictions are annotated if they occur in the validation files. The runner checks that the split hashes equal those recorded in Phase 3, that no split triple is a training triple, and that positives and negatives do not overlap. With TP the predictions that are positives, FP those that are supplied negatives, FN the positives not

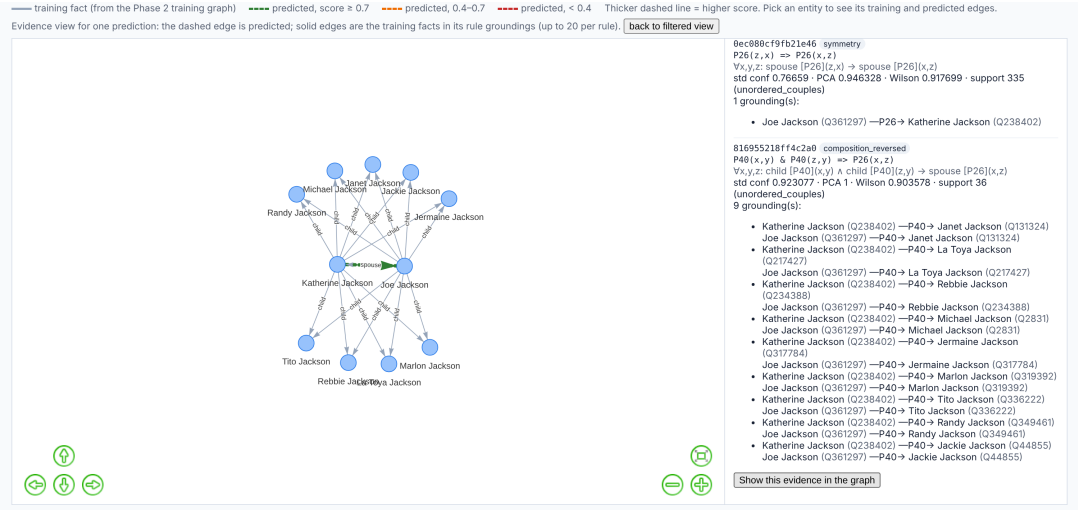


Figure 4.7: Evidence view and provenance panel for the prediction (Katherine Jackson, spouse, Joe Jackson): both supporting rules with their statistics and the training groundings drawn as solid edges.

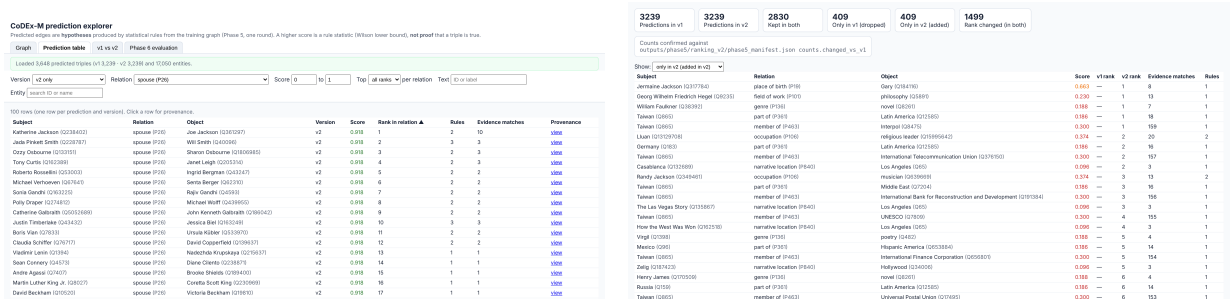


Figure 4.8: Prediction table filtered to V2 and spouse (P26), sorted by rank (left), and the V1-versus-V2 comparison tab showing 2,830 predictions kept in both, 409 dropped, 409 added and 1,499 with a changed rank (right).

predicted and P the number of predictions:

$$\text{Benchmark precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{|\text{positives}|}, \tag{4.5}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}, \qquad \text{Positive hit rate} = \frac{TP}{P}. \tag{4.6}$$

Benchmark precision is computed on the labelled subset only; it is *not* accuracy over all predictions. The positive hit rate is a lower bound on the share of correct predictions because unlabelled predictions may be true. The runner also reports specificity, labelled coverage $(TP + FP)/P$ and the recall ceiling under the cap, $\sum_r \min(P_r, |\text{positives}_r|)/|\text{positives}|$. Ranking metrics such as MRR or Hits@K are not reported, because the predictions are capped lists per relation, not answers to per-query rankings. The test evaluation is gated: it runs only with the flag `--i-approve-phase6-test-evaluation`.

4.11 Cap experiment and validation-based selection

At cap 100 the cut is decided by entity IDs inside tie groups in 31 (V1) and 32 (V2) of the 32 capped relations, and 7,681 test positives lie beyond the cap in their relation. `cap_experiment.py` therefore compares caps of 100, 250, 500 and 1,000 for both versions (eight configurations), in

four gated steps that each refuse to overwrite outputs:

1. `freeze-rule` writes the selection rule **A1** before any validation result exists (`selection_rule.json`, frozen 2026-10-29 06:55 UTC, SHA-256 recorded);
2. `generate` produces V1 and V2 predictions for caps 250, 500 and 1,000 with the same frozen rules, candidates, scores and tie-breaks. The cap-100 baseline is read from `outputs/phase5/` and is never rewritten; the driver first verifies that it reproduces the existing V2 baseline byte for byte, and checks the baseline hashes before and after;
3. `validate` labels all eight configurations against the validation split, with test files blocked, and applies rule A1;
4. `test --i-approve-cap-test-evaluation` evaluates only the selected configuration on the test split.

Rule A1 maximises validation recall (equivalently TP, since the denominator is the same for every configuration) subject to a precision floor: the Wilson 95% lower bound of validation benchmark precision must be at least the same bound for the cap-100 configuration of the same version minus 0.05. Ties go to the smaller cap, then to V2. Validation was already used in Phase 4, so its scores are optimistic; it is used here only to choose a configuration.

5 Implementation, Experiments and Evaluation

5.1 Tools and implementation status

The pipeline is written in Python 3.10. Phases 3–6, the cap experiment and both interface builders use only the Python standard library; Phase 2 uses NetworkX (graph) and pyvis (viewer). The interfaces are static HTML and JavaScript pages that work from `file://` and reuse the vis-network 9.1.2 library embedded in the Phase 2 viewer. Every script records the SHA-256 hashes of its inputs and outputs in a manifest, refuses to overwrite existing outputs, and has a unit-test module (152 Python test functions in `tests/`, plus JavaScript filter tests and a headless-browser smoke test). Table 5.1 gives the status of each stage.

Table 5.1: Implementation status of the pipeline stages.

Stage	Script	Main outputs	Status
Dataset inspection	(inspection report)	`phase1_dataset_report.md`	Completed
Training graph and viewer	`phase2_graph.py`	GraphML, statistics, `interactive_graph.html`	Completed
RDF/OWL ontology	`phase3_ontology.py`	`codexm_train_ontology.trig`, manifest	Completed
Rule mining	`phase4_rule_mining.py`	`run_default/`, `run_grid_ext/`	Completed
Validation rule review	`phase4_rule_evaluation.py`, `phase4_compare_runs.py`	160-rule list	Completed
Candidates and V1 ranking	`phase5_predict.py`	`outputs/phase5/predictions.json`	Completed
V2 ranking	`phase5_rank_v2.py`	`outputs/phase5/ranking_v2/`	Completed
Prediction explorer	`phase5_ui_build.py`	`outputs/prediction_ui/`	Completed
Test evaluation, cap 100	`phase6_evaluate.py`	`outputs/phase6/evaluation_test/`	Completed
Cap experiment	`cap_experiment.py`	`outputs/cap_experiment/`	Completed
Cap-comparison dashboard	`cap_dashboard_build.py`	`outputs/cap_comparison_dashboard/`	Completed
Ontology-aware scoring; MRR/Hits@K; baselines	–	–	Planned

5.2 Rule mining results

The rule miner (version 4B.2.0, `run_grid_ext`) scored 991 rule candidates with support of at least 5; 212 passed all thresholds and 194 were selected after redundancy and top-k filtering, 12 of them flagged as hub-dominated. The stability check selected 181 rules on the 90% split with a Jaccard overlap of 0.85; five rules were unstable. On the validation split the frozen `run_default` rules at the chosen floor cover 2,455 of 10,310 positives and 594 of 10,310 supplied negatives (coverage precision 0.805, recall 0.238); these are coverage figures of the rule bodies, not prediction metrics, and they were computed on the split that was used to choose the thresholds. After the review,

160 rules for 33 head relations were frozen.

5.3 Candidate generation results

The 160 rules produced 184,487 unique candidates for 33 relations. With a cap of 100 per relation the baseline keeps 3,239 predictions (32 relations reach the cap; child (P40) has only 39 candidates). Of these, 1,396 are supported by more than one rule, 143 of the 160 rules support at least one prediction, and the score tiers are 778 (≥ 0.7), 1,036 (0.4–0.7) and 1,425 (< 0.4); V2 has the same tiers because it reorders only inside equal-score groups. The candidate pools are very unequal: country of citizenship (P27) has 32,283 candidates, residence (P551) 20,713 and place of death (P20) 19,928, while child (P40) has 39.

5.4 Baseline test evaluation at cap 100 (Phase 6)

The cap-100 V1 and V2 predictions were evaluated on the test split in Phase 6 before the cap experiment was designed (Table 5.2). V2 recovers 21 more test positives with the same 13 supplied-negative matches. The Wilson 95% intervals of benchmark precision (V1 91.94–97.16%, V2 92.51–97.37%) overlap almost fully, so the precision difference is not meaningful; the gain is concentrated in member of (P463, +14 matches), languages spoken (P1412, +3) and part of (P361, +3). Four symmetric or structural relations (P26, P530, P3373, P495) give 141 of the matches in both versions. The recall ceiling under the cap is 23.94%.

Measure	**Cap 100 V1**	**Cap 100 V2**
Predictions	3,239	3,239
Positive test matches (TP)	257	278
Supplied-negative matches (FP)	13	13
Unlabelled predictions	2,969	2,948
of which in validation positives / negatives (information only)	246 / 12	253 / 11
Benchmark precision TP/(TP+FP)	95.19%	95.53%
Recall TP/10,311	2.49%	2.70%
F1 2TP/(2TP+FP+FN)	4.86%	5.24%
Positive hit rate TP/predictions	7.93%	8.58%
Specificity	99.87%	99.87%
Recall ceiling under the cap	23.94%	23.94%

Table 5.2: Test evaluation of the cap-100 baseline (`outputs/phase6/evaluation_test/evaluation_results.json`).

5.5 Cap experiment on the validation split

Table 5.3 and Figure 5.1 give the validation results of all eight configurations. Every configuration meets the precision floor of rule A1, so the rule selects the configuration with the most validation matches: **cap 1,000, V2** (875 matches, recall 8.49%, benchmark precision 93.38%, Wilson lower bound 91.61% against a floor of 87.69%). V2 has more matches than V1 at every cap and equal or fewer supplied-negative matches.

The cap-comparison dashboard (`outputs/cap_comparison_dashboard/index.html`) presents the same results from the saved files and shows the hash of every source file. Figure 5.2 shows its summary cards, validation charts and validation table.

Config.	Pred.	TP	FP	Unlab.	Prec.	Prec. lower	Recall	F1	Hit rate
100 V1	3,239	246	12	2,981	95.35%	92.05%	2.39%	4.66%	7.59%
100 V2	3,239	253	11	2,975	95.83%	92.69%	2.45%	4.79%	7.81%
250 V1	7,464	428	21	7,015	95.32%	92.96%	4.15%	7.96%	5.73%
250 V2	7,464	477	19	6,968	96.17%	94.09%	4.63%	8.83%	6.39%
500 V1	13,571	600	40	12,931	93.75%	91.60%	5.82%	10.96%	4.42%
500 V2	13,571	653	38	12,880	94.50%	92.54%	6.33%	11.87%	4.81%
1,000 V1	23,772	816	62	22,894	92.94%	91.05%	7.91%	14.59%	3.43%
1,000 V2	**23,772**	**875**	**62**	**22,835**	**93.38%**	**91.61%**	**8.49%**	**15.56%**	**3.68%**

Table 5.3: Validation comparison of the eight configurations (`outputs/cap_experiment/validation/validation_results.json`); recall over 10,310 validation positives. Prec. lower = Wilson 95% lower bound. Highlighted: the configuration selected by rule A1.

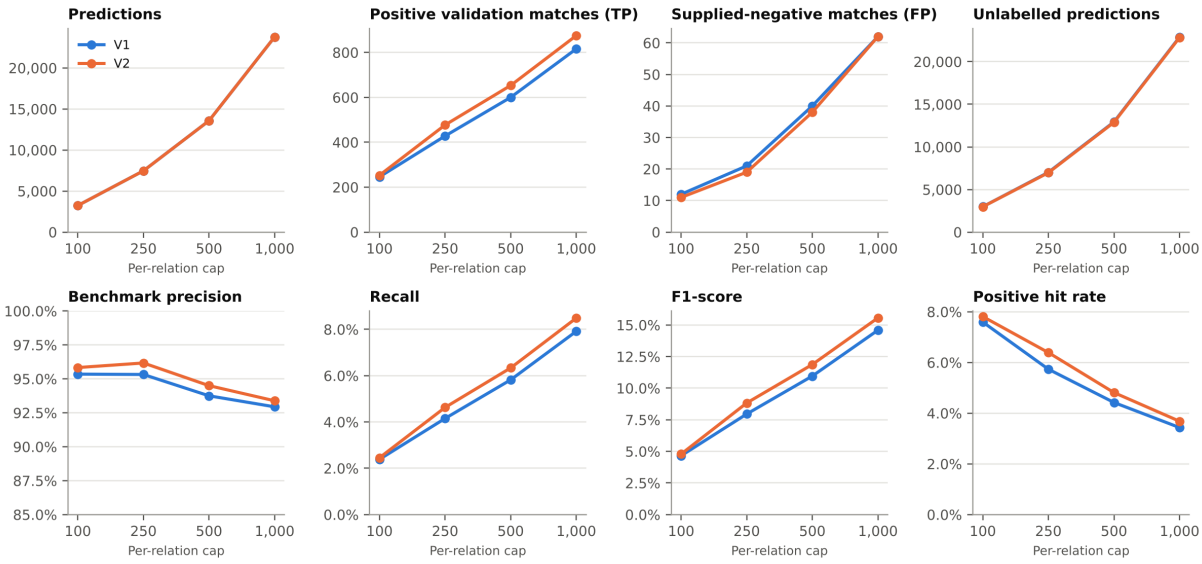


Figure 5.1: Validation metrics across per-relation caps for V1 and V2, plotted from `validation_results.json`. Benchmark precision falls slightly and the positive hit rate falls as the cap grows, while matches, recall and F1 rise.

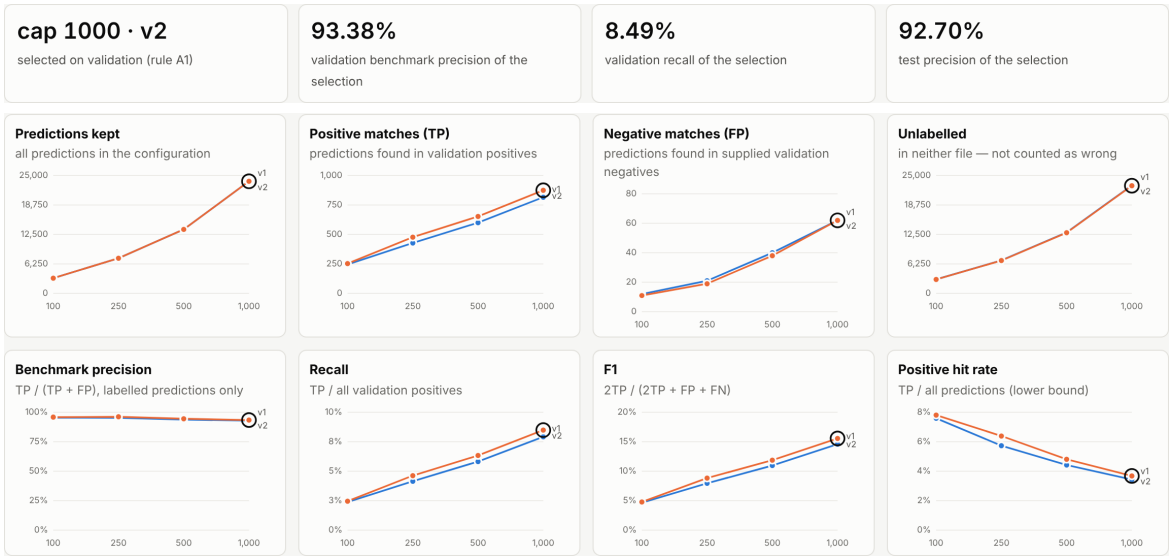
5.6 Final test evaluation of the selected configuration

The selected configuration was then evaluated once on the test split (`test_final/test_results.json`, generated 2026-10-29 07:21 UTC). Table 5.4 compares it with the cap-100 baseline; Figures 5.3 and 5.4 show the same numbers.

The 927 matches are concentrated in a few relations (Figure 5.5): diplomatic relation (P530) alone contributes 256 (with 2 supplied-negative matches), followed by member of (P463, 97), country of citizenship (P27, 91, with 14 supplied-negative matches) and languages spoken (P1412, 85). Spouse (P26) reaches 50 of its 53 test positives with 496 predictions, because its candidate pool is smaller than the cap.

5.7 Discussion

Trade-off of larger caps. Raising the cap from 100 to 1,000 (V2) multiplies the predictions by 7.3 and the test matches by 3.3 (278 to 927), so recall rises from 2.70% to 8.99% and F1 from



Configuration	Predictions	Relations still capped	Positive matches (TP)	Negative matches (FP)	Unlabelled	Precision	Precision 95% lower	Floor for this version	Recall	F1	Positive hit rate	Meets rule
cap 100 · v1	3,239	32	246	12	2,981	95.35%	92.05%	87.05%	2.39%	4.66%	7.59%	yes
cap 100 · v2	3,239	32	253	11	2,975	95.83%	92.69%	87.69%	2.45%	4.79%	7.81%	yes
cap 250 · v1	7,464	26	428	21	7,015	95.32%	92.96%	87.05%	4.15%	7.96%	5.73%	yes
cap 250 · v2	7,464	26	477	19	6,968	96.17%	94.09%	87.69%	4.63%	8.83%	6.39%	yes
cap 500 · v1	13,571	22	600	40	12,931	93.75%	91.60%	87.05%	5.82%	10.96%	4.42%	yes
cap 500 · v2	13,571	22	653	38	12,880	94.50%	92.54%	87.69%	6.33%	11.87%	4.81%	yes
cap 1,000 · v1	23,772	17	816	62	22,894	92.94%	91.05%	87.05%	7.91%	14.59%	3.43%	yes
cap 1,000 · v2 ◉ selected	**23,772**	**17**	**875**	**62**	**22,835**	**93.38%**	**91.61%**	**87.69%**	**8.49%**	**15.56%**	**3.68%**	**yes**

Figure 5.2: Screenshots of the cap-comparison dashboard: summary of the selected configuration (top), validation charts for caps 100–1,000 with the selected configuration ringed (middle), and the complete validation comparison table (bottom).

5.24% to 16.39%. Supplied-negative matches rise from 13 to 73, so benchmark precision falls from 95.53% to 92.70%, and because most added predictions are unlabelled the positive hit rate halves (8.58% to 3.90%). The same pattern holds on validation for both versions (Figure 5.1). A larger cap therefore trades a small loss of labelled precision and a larger loss of hit rate for more recovered positives; which side matters more depends on the use of the predictions.

Meaning of the numbers. Benchmark precision is measured on 1,000 of 23,772 predictions (4.2%); the other 22,772 are neither confirmed nor refuted, and the supplied negatives were not independently verified. The positive hit rate is only a lower bound on correct predictions. Recall is bounded by the cap and by the 11 relations without rules (162 test positives), and observed recall is about one eighth of the 71.97% ceiling. The results describe this rule set on this benchmark; they are not evidence of superiority over published methods.

Test-set methodology. The cap-100 baseline was evaluated on the test split in Phase 6, before the cap experiment. The selected cap-1,000 V2 configuration was afterwards evaluated on the same test split. The selection was made on validation with a rule frozen beforehand, and no threshold, rule, ranking or cap was changed after either test evaluation. Nevertheless this is a second use of the test split, so the test set is not an untouched hold-out across the project's history, and the final numbers must not be used to choose another configuration. Validation itself was used twice (Phase 4 rule review and cap selection), so validation scores are optimistic.

Answers to the research questions. *RQ1:* with one- and two-atom rules and a cap of 1,000, 927 of 10,311 test positives (8.99%) are recovered, at a benchmark precision of 92.70%.

Metric	Cap 100 V1	Cap 100 V2	Cap 1,000 V2
Predictions	3,239	3,239	23,772
Positive test matches (TP)	257	278	927
Supplied-negative matches (FP)	13	13	73
Unlabelled predictions	2,969	2,948	22,772
Benchmark precision	95.19%	95.53%	92.70%
Recall	2.49%	2.70%	8.99%
F1-score	4.86%	5.24%	16.39%
Positive hit rate	7.93%	8.58%	3.90%
Specificity	99.87%	99.87%	99.29%
Labelled coverage	8.34%	8.98%	4.21%
Recall ceiling under the cap	23.94%	23.94%	71.97%

Table 5.4: Test results of the cap-100 baseline (Phase 6) and of the selected cap-1,000 V2 configuration (second use of the test split).

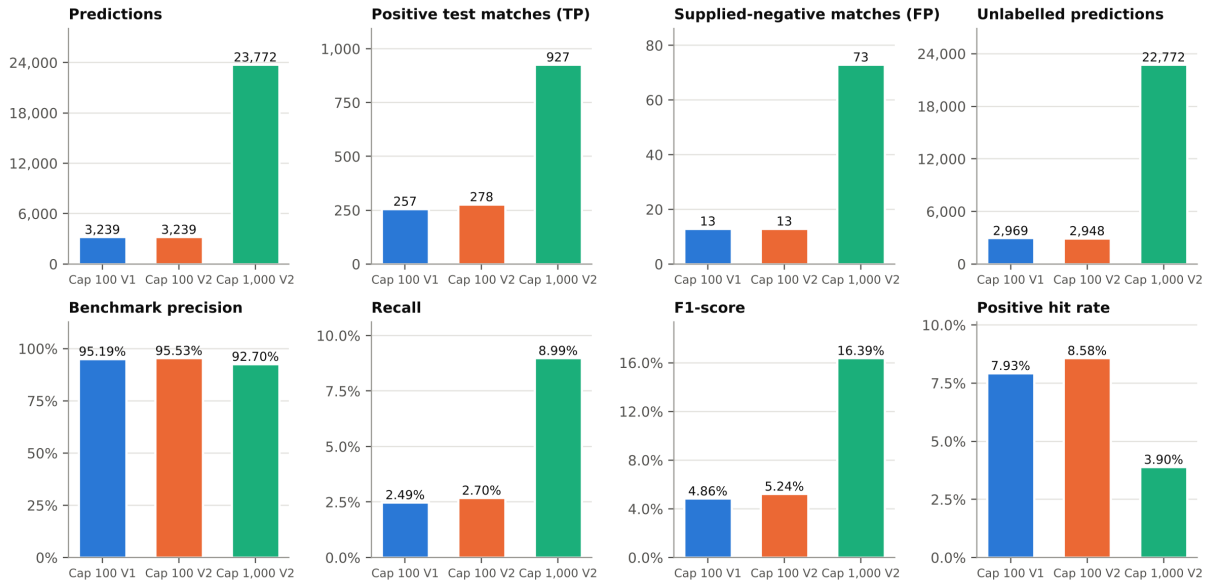


Figure 5.3: Test-split comparison of the three evaluated configurations, plotted from `evaluation_results.json` (cap 100) and `test_results.json` (cap 1,000 V2).

RQ2: the evidence tie-break (V2) adds matches at every cap on validation and 21 matches at cap 100 on test, with equal or fewer supplied-negative matches, although the precision difference is within overlapping intervals and the tie-break can favour hub entities. *RQ3:* larger caps raise recall and F1 and lower benchmark precision and hit rate; the main limits are the global per-rule score, rule coverage, ties broken by IDs, hub-driven candidates, the unused ontology and the absence of ranked metrics and baselines.

5.8 Verification and reproducibility

All counts in this chapter were read from the saved result files and checked for internal consistency (TP + FP + unlabelled = predictions for every configuration). The pipeline itself verifies its inputs: Phase 4 checks the TriG hash against the Phase 3 manifest, Phase 5 checks the GraphML source hash and the rule list hash, the cap experiment checks that its V2 driver reproduces the

Configuration	Predictions	Positive test matches (TP)	Supplied-negative matches (FP)	Unlabelled	Precision	Recall	F1	Positive hit rate
cap 100 · v1 (Phase 6 baseline)	3,239	257	13	2,969	95.19%	2.49%	4.86%	7.93%
cap 100 · v2 (Phase 6 baseline)	3,239	278	13	2,948	95.53%	2.70%	5.24%	8.58%
cap 1000 · v2 (selected)	23,772	927	73	22,772	92.70%	8.99%	16.39%	3.90%

Figure 5.4: Test table of the cap-comparison dashboard; the values agree with Table 5.4.

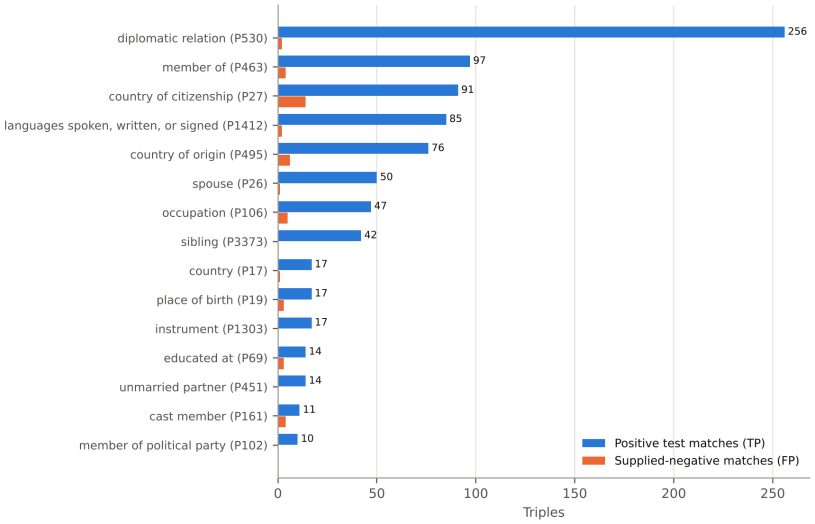


Figure 5.5: Positive and supplied-negative test matches of cap 1,000 V2 for the 15 relations with the most matches (`test_final/per_relation_metrics.csv`).

existing cap-100 V2 outputs byte for byte and that baseline files are unchanged before and after generation, and the evaluators check split hashes and overlaps. The dashboard's manifest lists the SHA-256 of each source file, so its displayed numbers can be traced to the experiment outputs.

6 Timeline, Milestones and Deliverables

6.1 Gantt chart

The project started at the beginning of July 2026; W1 is the week of 29 June 2026 and W18 the week of 26 October 2026. Tasks with dated project files are placed from the generation timestamps recorded in their manifests (dark green). The literature review, the problem formulation and the earlier FB15K-237 prototype have no dated artefact in the current repository, so their bars are approximate (light green) and follow the previous draft of this report. The earlier prototype was superseded by the CoDEx-M implementation and none of its results is used here. Planned work is shown in relative months (m1–m6) whose dates need confirmation with the guide; no completion dates are asserted for it.

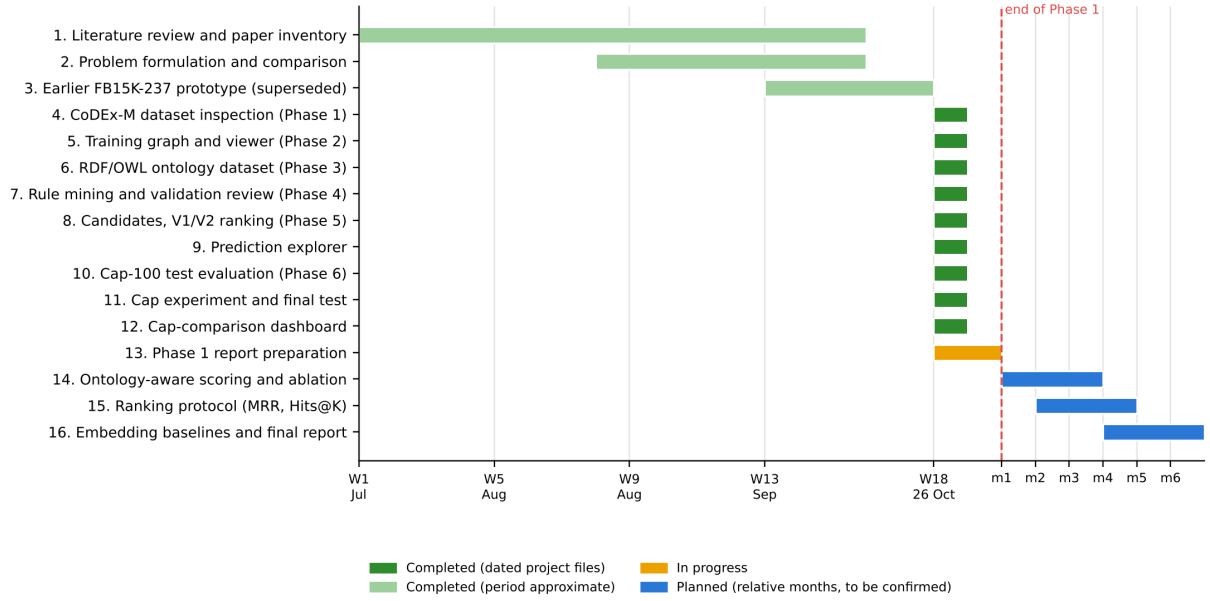


Figure 6.1: Gantt chart. Dark green: completed, placed from dated project files; light green: completed, period approximate; amber: in progress; blue: planned (relative months, dates to be confirmed). The red line marks the end of Phase 1.

Table 6.1: Gantt tasks, status and the dated evidence used to place them (times in UTC, from the manifests).

No.	Task	Status	Evidence
1–3	Literature review; problem formulation; earlier prototype	Completed (approx.)	Paper archive files dated August–October 2026; previous report draft
4	Dataset inspection	Completed	`phase1_dataset_report.md`, inspection date 28 Oct 2026
5	Training graph and viewer	Completed	`graph_statistics.json`, 28 Oct 2026 17:09
6	RDF/OWL ontology	Completed	`phase3_manifest.json`, 28 Oct 2026 18:11
7	Rule mining and validation review	Completed	Mining manifests 18:43 and 19:07; validation evaluation 18:44, 28 Oct 2026

No.	Task	Status	Evidence
8	Candidates and V1/V2 ranking	Completed	Phase 5 manifests 19:30 (V1) and 19:37 (V2), 28 Oct 2026
9	Prediction explorer	Completed	`ui_manifest.json`, 29 Oct 2026 06:13
10	Cap-100 test evaluation	Completed	`evaluation_manifest.json`, 29 Oct 2026 05:53
11	Cap experiment and final test	Completed	Rule frozen 06:55, selection 06:57, test 07:21, 29 Oct 2026
12	Cap-comparison dashboard	Completed	`dashboard_manifest.json`, 29 Oct 2026 07:23
13	Phase 1 report	In progress	This report; guide review pending
14–16	Ontology-aware scoring; ranking protocol; baselines and final report	Planned	Section 7.3

6.2 Milestones and deliverables

Table 6.2 lists the milestones, which agree with the objectives (Table 3.1) and the Gantt chart (Figure 6.1). M1–M7 are Phase 1 achievements verified from the saved artefacts, M8 is in progress, and M9–M11 are future deliverables with relative target periods.

Table 6.2: Milestones and deliverables.

ID	Milestone	Deliverable	Completion criterion	Obj.	Status
M1	Literature base	43 papers, thematic review, gap analysis	All papers inventoried and cited; duplicates removed	–	Completed
M2	Verified data	Phase 1 report	Counts and overlaps checked	O1	Completed
M3	Graph and ontology	GraphML, viewer, TriG dataset	Counts verified; reload and allowlist checks pass	O2, O3	Completed
M4	Rules	160 frozen rules with statistics	Validation review documented	O4	Completed
M5	Predictions	V1/V2 predictions, caps 100–1,000	Hash-checked manifests; baseline preserved	O5	Completed
M6	Interfaces	Prediction explorer, cap dashboard	Values traceable to source files	O6	Completed
M7	Evaluation	Phase 6 results, validation selection, final test	Frozen selection rule; gated test runs	O7	Completed
M8	Phase 1 report	This report, architecture diagram, figures	Guide's review	All	In progress
M9	Ontology-aware scoring	Consistency factor with ablation	Validation comparison with the baseline	O3, O5	Planned (m1–m3)
M10	Ranking protocol	MRR, Hits@K evaluator	Defined candidate universe and tie policy	O7	Planned (m2–m4)
M11	Baselines and final report	Embedding baseline table; final report	Same splits and metrics	O7	Planned (m4–m6)

The Phase 1 deliverables are the pipeline and its saved outputs (graph, ontology, rules, predictions, explorer, dashboard, evaluation files and manifests), the verified results of Chapter 5, the architecture diagram with its editable source, and this report.

7 Conclusion, Limitations and Future Work

7.1 Conclusion

Literature and research gap. The review of 43 unique papers, grouped by research theme and including the five selected papers, shows that rule discovery and confidence estimation are well studied but mostly global: query-dependent scoring [4], probabilistic combination of rules [5], PCA confidence [2] and path explanations [10] address parts of the problem, ontology semantics enter completion mainly inside neural models [7], and evaluation mostly uses filtered ranking. What remains open is a transparent candidate score that combines rule reliability, candidate-specific evidence and a semantic-consistency check, evaluated under the open-world assumption.

Implemented system. On CoDEx-M the project verifies the splits, builds a relation-labelled training graph and an RDF/OWL dataset with separate assertion, label and provenance graphs, mines one- and two-atom rules with PCA confidence and Wilson bounds, freezes 160 rules after a validation review, generates 184,487 candidates from training facts only, ranks them with two documented tie-break policies, explains every prediction by its rules and groundings, and presents the results in a prediction explorer and a cap-comparison dashboard.

Evidence. At cap 100 the V2 ranking recovers 278 test positives (benchmark precision 95.53%, recall 2.70%) against 257 for V1. The configuration selected on validation, cap 1,000 with V2, recovers 927 test positives with 73 supplied-negative matches (benchmark precision 92.70%, recall 8.99%, F1 16.39%, positive hit rate 3.90%); 22,772 of its predictions are unlabelled and are treated as unknown.

7.2 Limitations

- The test split was used twice (cap-100 baseline, then the selected configuration), and validation was used both for the rule review and for cap selection.
- The score is a global per-rule statistic; candidate-specific evidence is used only to break ties, and scores are not calibrated probabilities.
- The ontology is generated but carries no domain, range or class axioms and does not influence mining or scoring.
- Rules have at most two body atoms and no constants; 11 relations with test positives receive no prediction, and six hub bodies were skipped.
- Ties are still broken by entity IDs at the cap boundary, and the evidence tie-break can favour hubs.
- Supplied negatives were checked only structurally; most predictions are unlabelled, so benchmark precision covers a small labelled subset.
- No MRR, Hits@K, embedding or neural baseline has been run, so no comparison with published methods is possible.
- The explanations have not been evaluated with users.

7.3 Future work

1. **Ontology-aware scoring.** Derive verified relation domains and ranges (for example from observed endpoint patterns reviewed against an authoritative source), add a consistency factor to the score, and keep it only if an ablation on validation shows a gain.
2. **Evidence-aware scoring.** Move distinct body matches, with a hub penalty, from the tie-break into the score, and compare noisy-OR and probabilistic combination [5].
3. **Coverage.** Add longer rules, rules with constants and object identity [1].
4. **Ranking protocol.** Define per-query candidate universes and a tie policy [4] to report filtered MRR and Hits@K.
5. **Baselines.** Train embedding baselines (TransE, DistMult, ComplEx, RotatE [17–20]) on the same splits.
6. **Clean evaluation.** Freeze all Phase 2 choices on validation and report one final evaluation; consider a fresh held-out sample for annotation of unlabelled predictions.

Ontology-aware scoring will be claimed to help only if an ablation shows it.

References

The references are numbered in the order of discussion in Chapter 2 (most relevant themes first) and grouped by research theme. References [1]–[43] are the 43 unique papers of the supplied archive; † marks the five selected papers. Reference [44] is the dataset source and is not part of the reviewed archive. Entries marked as preprints have not been verified as peer-reviewed.

1. Rule-Based Knowledge Graph Completion and Logical Rule Learning

[1] C. Meilicke, M. W. Chekol, P. Betz, M. Fink, and H. Stuckenschmidt, "Anytime bottom-up rule learning for large-scale knowledge graph completion," *The VLDB Journal*, vol. 33, pp. 131–161, 2024, doi: 10.1007/s00778-023-00800-5.

[2] L. Wu, E. Sallinger, E. Sherkhonov, S. Vahdati, and G. Gottlob, "Rule learning over knowledge graphs with genetic logic programming," in *Proc. IEEE 38th Int. Conf. Data Engineering (ICDE)*, pp. 3373–3385, 2022, doi: 10.1109/ICDE53745.2022.00318. †

[3] P. Ghiasnezhad Omran, K. Taylor, S. Rodríguez Méndez, and A. Haller, "Active knowledge graph completion," *Information Sciences*, vol. 604, pp. 267–279, 2022, doi: 10.1016/j.ins.2022.05.027.

2. Rule Confidence and Candidate-Specific Scoring

[4] T. H. Le, T. C. Son, I. Ahmed, and H. Cao, "SLogic: Subgraph-informed logical rule learning for knowledge graph completion," arXiv preprint arXiv:2510.00279 (v3), 2025. †

[5] J. M. Patil, N. Lee, A. M. S. Chowdhury, Y. Choi, and P. Shakarian, "Probabilistic circuits for knowledge graph completion with reduced rule sets," arXiv preprint arXiv:2508.06706 (v2), 2025. †

3. Ontology-Based Reasoning and Semantic Constraints

[6] W. Zhang, J. Chen, J. Li, Z. Xu, J. Z. Pan, and H. Chen, "Knowledge graph reasoning with logics and embeddings: survey and perspective," in *Proc. IEEE Int. Conf. Knowledge Graph (ICKG)*, pp. 492–499, 2024, doi: 10.1109/ICKG63256.2024.00069.

[7] J. Wang, W. Li, X. Zhang, Q. Hu, A. M. Luvembe, R. Liu, F. Liu, and Q. Jin, "Ontology-enhanced subgraph reasoning with prompt learning for inductive knowledge graph completion," *Expert Systems with Applications*, vol. 320, art. 131944, 2026, doi: 10.1016/j.eswa.2026.131944.

[8] X. Lv, L. Hou, J. Li, and Z. Liu, "Differentiating concepts and instances for knowledge graph embedding," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1971–1979, 2018.

[9] T. S. Balakrishnan, L. Karthikeyan, U. S. Ebenezar, and B. S. Preetham, "Ontology-driven semantic interoperability framework for multi-center healthcare big data with graph-based query analysis," in *Proc. Int. Conf. Computing for Sustainability and Intelligent Future (COMP-SIF)*, 2025, doi: 10.1109/COMP-SIF65618.2025.10969920.

4. Explainable Knowledge Graph Inference

[10] H. Yu, H. Yu, W. Wang, K. Fang, X. Zhang, and H. Liu, "An explainable path reasoning framework for knowledge graph completion," *Alexandria Engineering Journal*, vol. 134, pp. 321–328, 2026, doi: 10.1016/j.aej.2025.12.024.

[11] X. Li, Q. Liu, Q. Bai, and X. Xu, "The role of knowledge graphs on responsible artificial intelligence realization: research opportunities and challenges," *Computer (IEEE)*, 2025, doi: 10.1109/MC.2025.3545036.

5. Neural-Symbolic Reasoning and Differentiable Rule Learning

[12] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, arXiv:1702.08367. †

[13] M. Qu, J. Chen, L.-P. Xhonneux, Y. Bengio, and J. Tang, "RNNLogic: learning logic rules for reasoning on knowledge graphs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021, arXiv:2010.04029.

[14] K. Cheng, N. K. Ahmed, and Y. Sun, "Neural compositional rule learning for knowledge graph reasoning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2303.03581.

[15] P. Sen, B. W. S. R. de Carvalho, I. Abdelaziz, P. Kapanipathi, S. Roukos, and A. Gray, "Logical neural networks for knowledge base completion with embeddings and rules," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3863–3875, 2022.

[16] X. Zhang, Z. Wang, G. Rao, and K. Wang, "Implicit and explicit rule injection for complex query answering over knowledge graphs," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2025, doi: 10.1109/ICASSP49660.2025.10889593.

6. Knowledge Graph Embeddings and Link Prediction

[17] B. Yang, W. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015, arXiv:1412.6575.

[18] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, arXiv:1606.06357.

[19] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "RotatE: knowledge graph embedding by relational rotation in complex space," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019, arXiv:1902.10197.

[20] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Advances in Neural Information Processing Systems 26 (NIPS 2013)*, 2013.

[21] Z. Wang, J. Zhang, J. Feng, and Z. Chen, "Knowledge graph embedding by translating on hyperplanes," in *Proc. 28th AAAI Conf. Artificial Intelligence*, 2014.

[22] Y. Lin, Z. Liu, M. Sun, Y. Liu, and X. Zhu, "Learning entity and relation embeddings for knowledge graph completion," in *Proc. 29th AAAI Conf. Artificial Intelligence*, 2015.

[23] M. Fan, Q. Zhou, E. Chang, and T. F. Zheng, "Transition-based knowledge graph embedding with relational mapping properties," in *Proc. 28th Pacific Asia Conf. Language, Information and Computation (PACLIC)*, pp. 328–337, 2014.

7. Graph Neural Networks and Advanced Learning Methods

[24] Z. Shang, P. Wang, J. Liu, J. Liu, G. Li, Z. Xu, Z. Luo, X. Chen, and W. Ke, "Learning multi-granularity and adaptive representation for knowledge graph reasoning," *IEEE Trans. Knowledge and Data Engineering*, vol. 37, no. 9, 2025, doi: 10.1109/TKDE.2025.3579774.

[25] K. Yang, H. Gao, and Y. Yang, "Entities and relations aware graph convolutional network for knowledge base completion," in *Proc. IEEE 9th Int. Conf. Information, Communication and Networks (ICICN)*, 2021, doi: 10.1109/ICICN52636.2021.9673867.

[26] Q. Wu and Q. He, "Knowledge graph embedding based graph convolutional network for link prediction," in *Proc. 11th Int. Conf. Advanced Cloud and Big Data (CBD)*, 2023, doi: 10.1109/CBD63341.2023.00050.

[27] Q. Rao and T. Wang, "Semantic enhancement based knowledge graph completion for graph convolutional neural networks," in *Proc. 7th Int. Conf. Electrical, Mechanical and Computer Engineering (ICEMCE)*, 2023, doi: 10.1109/ICEMCE60359.2023.10490589.

[28] Z. Ye, Y. J. Kumar, G. O. Sing, F. Song, and J. Wang, "A comprehensive survey of graph neural networks for knowledge graphs," *IEEE Access*, vol. 10, 2022, doi: 10.1109/ACCESS.2022.3191784.

[29] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions," *Journal of Big Data*, vol. 11, art. 18, 2024, doi: 10.1186/s40537-023-00876-4.

[30] M. Kharbanda, R. R. Shah, and R. Mutharaju, "RConE: rough cone embedding for multi-hop logical query answering on multi-modal knowledge graphs," *IEEE Trans. Knowledge and Data Engineering*, vol. 37, no. 10, pp. 6135–6146, 2025, doi: 10.1109/TKDE.2025.3584054.

[31] Y. Liu, D. Chen, Y. Fu, H. Bai, H. Cui, and R. Peng, "Process knowledge graph reasoning based on graph neural network," in *Proc. 37th Chinese Control and Decision Conf. (CCDC)*, 2025, doi: 10.1109/CCDC65474.2025.11090305.

[32] Y. Wang, Y. Chen, and J. Sun, "Fault prediction method for industrial Internet of Things based on knowledge graph," in *Proc. 5th Int. Conf. Machine Learning and Intelligent Systems Engineering (MLISE)*, 2025, doi: 10.1109/MLISE66443.2025.11100161.

8. Knowledge Graph Surveys, Benchmarks and Evaluation

[33] H. Liu, S. Yang, G. Shi, and Z. Miao, "Knowledge graph reasoning: mainstream methods, applications and prospects," *Engineering Applications of Artificial Intelligence*, vol. 159, art. 111625, 2025, doi: 10.1016/j.engappai.2025.111625.

[34] G. Niu, B. Li, and Y. Lin, "A comprehensive survey of knowledge graph reasoning: approaches and applications," *IEEE Trans. Big Data*, vol. 12, no. 3, pp. 721–740, 2026, doi: 10.1109/TBDATA.2026.3668633.

[35] D. Li, Y. Liang, S. Qian, H. Wu, W. Jia, Y. Fu, and Y. Sun, "A review of background, methods, limitations and opportunities of knowledge graph completion," *Computer Science Review*, vol. 58, art. 100809, 2025, doi: 10.1016/j.cosrev.2025.100809.

[36] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: representation, acquisition, and applications," *IEEE Trans. Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022, doi: 10.1109/TNNLS.2021.3070843.

[37] Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge graph embedding: a survey of approaches and applications," *IEEE Trans. Knowledge and Data Engineering*, vol. 29, no. 12, pp. 2724–2743, 2017, doi: 10.1109/TKDE.2017.2754499.

[38] M. Wang, L. Qiu, and X. Wang, "A survey on knowledge graph embeddings for link prediction," *Symmetry*, vol. 13, no. 3, art. 485, 2021, doi: 10.3390/sym13030485.

[39] J. Cao, J. Fang, Z. Meng, and S. Liang, "Knowledge graph embedding: a survey from the perspective of representation spaces," *ACM Computing Surveys*, vol. 56, no. 6, art. 159, pp. 1–42, 2024, doi: 10.1145/3643806. †

[40] R. Biswas, L.-A. Kaffee, M. Cochez, S. Dumbrava, T. E. Jendal, M. Lissandrini, V. Lopez, E. Loza Mencía, H. Paulheim, H. Sack, E. Kalemi Vakaj, and G. de Melo, "Knowledge graph embeddings: open challenges and opportunities," *Transactions on Graph Data and Knowledge*, vol. 1, no. 1, art. 4, 2023, doi: 10.4230/TGDK.1.1.4.

[41] J. Sardina, J. D. Kelleher, and D. O'Sullivan, "A survey on knowledge graph structure and knowledge graph embeddings," in *Proc. 19th Int. Conf. Semantic Computing (ICSC)*, 2025, doi: 10.1109/ICSC64641.2025.00008.

[42] G. A. Gesese, R. Biswas, M. Alam, and H. Sack, "A survey on knowledge graph embeddings with literals: which model links better literal-ly?," *Semantic Web*, vol. 12, no. 4, pp. 617–647, 2021, doi: 10.3233/SW-200404.

[43] Z. Wang and A. Li, "Temporal knowledge graph reasoning: a survey of representation learning, rule-based learning, and neural-symbolic hybrid methods," in *Proc. 10th IEEE Int. Conf. Data Science in Cyberspace (DSC)*, 2025, doi: 10.1109/DSC67331.2025.00019.

Dataset source (not part of the reviewed archive)

[44] T. Safavi and D. Koutra, "CoDEx: A comprehensive knowledge graph completion benchmark," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8328–8350, 2020.

A Repository and Reproducibility Notes

This appendix summarises, from the project folder `KG-Ontology-Rule-Based-Project` and the README files in its output folders, how the saved results are produced and where they are stored.

A.1 Repository structure

```
KG-Ontology-Rule-Based-Project/
  data/                train.txt, valid.txt, test.txt, valid_negatives.txt, test_negatives.txt,
                       entity_labels.json, relation_labels.json
  display_labels.py    presentation-only ID-to-label helper (DISPLAY_LABELS.md)
  phase2_graph.py      training graph, statistics, interactive viewer
  phase3_ontology.py   RDF/OWL TriG dataset with train / labels / provenance named graphs
  phase4_rule_mining.py, phase4_rule_evaluation.py, phase4_compare_runs.py
  phase5_predict.py    candidates, scoring, ranking V1, per-relation cap
  phase5_rank_v2.py    ranking V2 (evidence tie-break)
  phase5_ui_build.py   prediction explorer builder      (sources: prediction_ui_src/)
  phase6_evaluate.py   gated test evaluation
  phase6_report.py     Phase 6 report and charts
  cap_experiment.py    caps 100/250/500/1000: freeze-rule, generate, validate, test
  cap_dashboard_build.py  cap-comparison dashboard builder (sources: cap_dashboard_src/)
  tests/               unit tests (Python) and ui/ (JavaScript filter and browser tests)
  outputs/
    phase1_dataset_report.md
    phase2/        training_graph.graphml, graph_statistics.json, interactive_graph.html, ...
    phase3/        codexm_train_ontology.trig, phase3_manifest.json, ontology_design.md
    phase4/        run_default/, run_grid_ext/, phase5_candidate_rules_proposed.json, reports
    phase5/        predictions.json (V1, cap 100), ranking_v2/ (V2, cap 100), viewers, manifests
    phase6/        evaluation_test/ (labels, metrics, per-relation CSV, report, charts)
    prediction_ui/ index.html and data/ (prediction explorer)
    cap_experiment/  cap_250/, cap_500/, cap_1000/ (v1, v2), validation/, test_final/,
                     selection_rule.json, selection.json, cap_experiment_report.md
    cap_comparison_dashboard/  index.html, data/cap_results.js, dashboard_manifest.json
```

A.2 Running the pipeline

From the project root (Windows, `py` launcher), each step writes to a new folder and refuses to overwrite existing outputs:

```
py phase2_graph.py
py phase3_ontology.py                # py phase3_ontology.py --verify-only rechecks outputs
py phase4_rule_mining.py --run-name <new run>
py phase4_rule_evaluation.py
py phase4_compare_runs.py
py phase5_predict.py --cap 100 --out-dir <new folder>
py phase5_rank_v2.py
py phase5_ui_build.py --out-dir outputs\prediction_ui_rebuild
py phase6_evaluate.py --i-approve-phase6-test-evaluation      # gated; already run once
py cap_experiment.py freeze-rule
py cap_experiment.py generate
py cap_experiment.py validate
py cap_experiment.py test --i-approve-cap-test-evaluation     # gated; already run once
py cap_dashboard_build.py
py -m pytest tests                   # or: py -m unittest
```

The two gated commands have already been run; running them again would be a further use of the test split and is not intended. The prediction explorer and the dashboard open by double-

clicking their `index.html`; if a browser blocks local scripts, `py -m http.server 8000` serves the project folder.

A.3 Output files used in this report

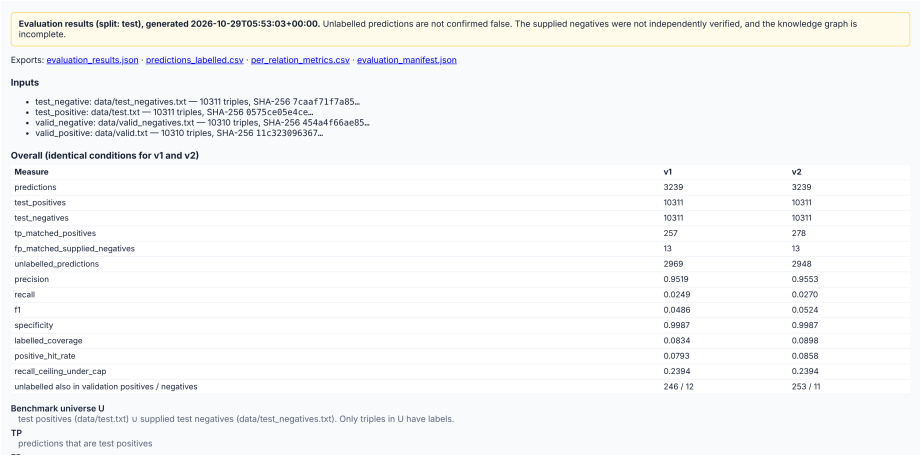
Table A.1: Output files used in this report and their content.

File	Content
`outputs/phase1_dataset_report.md`	Split statistics, format and overlap checks
`outputs/phase2/graph_statistics.json`	Node, edge and relation counts; relation frequencies; path observations
`outputs/phase2/interactive_graph.html`	Offline training-graph explorer (Figure 4.3)
`outputs/phase3/codexm_train_ontology.trig`	RDF/OWL dataset (Listing 4.1)
`outputs/phase3/phase3_manifest.json`	Named-graph counts, verification results, hashes
`outputs/phase4/run_grid_ext/selected_rules.json`	194 selected rules with statistics and examples
`outputs/phase4/phase5_candidate_rules_proposed.json`	Frozen 160-rule list and the six exclusions
`outputs/phase5/phase5_manifest.json`	Candidate counts and settings (V1, cap 100)
`outputs/phase5/ranking_v2/ranking_comparison.md`	V1–V2 differences per relation
`outputs/phase6/evaluation_test/evaluation_results.json`	Cap-100 V1/V2 test labels and metrics
`outputs/cap_experiment/selection_rule.json`	Selection rule A1 (frozen before validation)
`outputs/cap_experiment/validation/validation_results.json`	Validation metrics of all eight configurations
`outputs/cap_experiment/selection.json`	Selected configuration and eligibility table
`outputs/cap_experiment/test_final/test_results.json`	Final test metrics of cap 1,000 V2
`outputs/cap_experiment/test_final/per_relation_metrics.csv`	Per-relation test metrics (Figure 5.5)
`outputs/cap_comparison_dashboard/dashboard_manifest.json`	Source hashes of the dashboard numbers

A.4 Reproducibility measures

The data files are hashed in the Phase 3 manifest and rechecked by later phases; every phase writes its settings, input hashes and output hashes to a manifest; the builders refuse to overwrite existing outputs; the miners and predictors are tested to open no validation or test file; the cap experiment verifies that it reproduces the cap-100 baseline byte for byte and leaves it unchanged; and the two test evaluations are gated by explicit approval flags. The Phase 6 interface tab (Figure A.1)

displays the stored test results with their input hashes and never runs an evaluation.



Evaluation results (split: test), generated 2026-10-29T05:53:03+00:00. Unlabelled predictions are not confirmed false. The supplied negatives were not independently verified, and the knowledge graph is incomplete.

Exports: evaluation_results.json · predictions_labelled.csv · per_relation_metrics.csv · evaluation_manifest.json

Inputs

- test_negative: data/test_negatives.txt — 10311 triples, SHA-256 7caaf71f7a85…
- test_positive: data/test.txt — 10311 triples, SHA-256 0575ce05e4ce…
- valid_negative: data/valid_negatives.txt — 10310 triples, SHA-256 454a4f66ae85…
- valid_positive: data/valid.txt — 10310 triples, SHA-256 11c323096367…

Overall (identical conditions for v1 and v2)

Measure	v1	v2
predictions	3239	3239
test_positives	10311	10311
test_negatives	10311	10311
tp_matched_positives	257	278
fp_matched_supplied_negatives	13	13
unlabelled_predictions	2969	2948
precision	0.9519	0.9553
recall	0.0249	0.0270
f1	0.0486	0.0524
specificity	0.9987	0.9987
labelled_coverage	0.0834	0.0898
positive_hit_rate	0.0793	0.0858
recall_ceiling_under_cap	0.2394	0.2394
unlabelled also in validation positives / negatives	246 / 12	253 / 11

Benchmark universe U
test positives (data/test.txt) ∪ supplied test negatives (data/test_negatives.txt). Only triples in U have labels.

TP
predictions that are test positives

Figure A.1: Phase 6 evaluation tab of the prediction explorer: inputs with SHA-256 prefixes and the cap-100 V1/V2 test metrics.